Scripture Pipelines

Declarative AI Pipelines for Biblical and Linguistic Scholarship

Jonathan Robie Biblica Nida Institute jonathan.robie@biblica.com

Abstract

Biblical and linguistic scholarship now has more open data than it can use: morphological treebanks, discourse annotations, lexicographic databases, papyri, inscriptions — much of it freely licensed, most of it underused. The bottleneck is human capacity, not data. AI, used well, can act as a synthesis layer between structured scholarly datasets and usable scholarly products. The challenge is using it well: rigorously, reproducibly, and with the accountability that lets a scholar verify what the model actually did with the evidence it was given.

Scripture Pipelines is an open-source declarative pipeline system built for this problem. Rather than embedding AI calls in imperative code, Scripture Pipelines expresses each workflow as a YAML pipeline: a sequence of named steps, each with explicit inputs, outputs, and prompt contracts. Every intermediate result is persisted to disk. Every LLM step can be required to produce a structured audit record of the sources it used. This paper describes the design decisions behind Scripture Pipelines and reports on its application in several Nida Institute projects producing freely licensed biblical reference materials — including a multi-pass Greek semantic lexicon, a grammatical phrasing analysis of the Hebrew Bible, book-level discourse analyses, and leaders' guides for Bible study. We examine how Scripture Pipelines chooses the best markup for each purpose, how shared identifiers make format translation mechanical, and how its iteration mechanisms support the scholarly workflow of generate, inspect, and refine.

Introduction: The Kairos Project and Scripture Pipelines

Every software system is also a social system. It decides who has power, who is included, who is left out, how people communicate. Scripture Pipelines is built to serve The Kairos Project — a Nida Institute initiative to build a global community of scholars serving Bible translation and the global church. The workflow patterns described in this paper are in service of that goal: building collaboration around data-driven biblical scholarship, where each Scripture Pipelines project is meant to support its own communities.

The name is intentional. *Kairos* is the Greek word for the right time, the opportune moment — the moment when conditions align and action becomes possible. We believe this is that moment. The open data exists. The communities who need it exist. The capability to use AI to synthesize data at scale exists. The question is not whether we will respond to the opportunity, but how best to respond — how we will build the infrastructure and the collaborative practices that make rigorous, reproducible, AI-assisted scholarship possible, and how we will organize that work to serve not just the academy but the global church.

The Kairos Project exists to raise up scholars worldwide, working together to produce the kind of resources that are actually needed — not only what is useful for Western academics, but what translation teams, study groups, and readers in every culture can use. The goal is to fully equip the global church by producing high quality, freely licensed resources together with communities worldwide, resources that they can adapt to their own needs. This means building a scholarly community spanning the Western academy and far beyond it — scholars who want to serve Bible translation and the needs of readers worldwide. Part of what that means is providing information in the most useful form for communities that traditional scholarly resources were not built for: non-Western communities, oral cultures, people who engage Scripture outside the academy, and people learning Greek and Hebrew outside traditional seminaries — often multilingual learners whose linguistic gifts may not be accompanied by formal credentials. These communities need different formats than academic reference works typically provide, and they have never needed them packaged that way. This matters because the biblical texts were written by and for people immersed in ancient cultures and social worlds very different from any modern setting — and every modern reader, regardless of background, needs help understanding those foreign contexts. Most people today do not recline at tables, have never seen a hoopoe, and are unfamiliar with the honor dynamics of Mediterranean patronage systems. Scholarly insights bridge that gap. A journal article is the wrong way to present that information to most readers — there are simpler, more straightforward ways. Sometimes it looks like a map, or a picture, or a set of questions that help a group notice what is actually in the text. Sometimes it looks like knowing that a certain phrase carries a weight that the translation cannot fully render, or that a story is being told in a pattern that an ancient audience would have recognized immediately.

Scholarly resources have traditionally been packaged in forms that serve academic communities well — journal articles, monographs, reference works dense with technical terminology. But this creates real barriers for readers outside the academy, for oral cultures, for translation teams working in contexts where academic conventions may be unfamiliar, and for multilingual learners whose linguistic gifts may not be accompanied by formal credentials. The lack of resources in forms these communities can actually use has been a real bottleneck. The Kairos Project exists to open those doors — to serve the whole range of people who want to engage Scripture deeply: those learning to use this

technology to produce resources for their local churches, those learning to read Greek and Hebrew, those doing serious biblical scholarship, and those learning to lead local Bible studies. The goal is building interlocking communities, not just software. Each community needs different resources at different levels of depth, but they are not separate silos. Technical infrastructure serves language learners; scholars equip study leaders; insights from local reading communities flow back into scholarship and enrich it. These resources provide support at every level: from technical documentation for pipeline architects, to lexicons and discourse analyses for scholars, to study guides for group leaders reading Scripture together in community. The goal is communities that talk with each other, where each community learns from the others: scholarly insight goes out in forms communities can actually use, and the questions those communities bring reshape what scholarship attends to. Readers in community can encounter the text directly and make genuine discoveries. The scholar's role is to help inform that encounter, not to conduct it on the reader's behalf.

Scripture Pipelines is an infrastructure that makes this vision possible. It is an open-source declarative pipeline system designed to synthesize structured scholarly datasets at scale using AI, while maintaining the reproducibility, auditability, and transparency that scholarly work requires. Rather than embedding AI calls in imperative code, Scripture Pipelines expresses each workflow as a YAML pipeline: a sequence of named steps with explicit inputs, outputs, and prompt contracts. Each pipeline step declares a contract: what data it takes (and from where), what the prompt instructs the AI to do, and what output it promises to produce. Because every intermediate result is saved, scholars can test one step at a time without running the entire pipeline—and they can audit each step's work: which sources were actually used, whether the AI followed the prompt's instructions, and whether the result matches what was specified. When a prompt needs refinement, the same pipeline that generates a lexicon entry can rerun with the revised prompt without re-querying the corpus-gathering step. This paper describes Scripture Pipelines' design and reports on its application across multiple Nida Institute projects producing freely licensed biblical reference materials — lexicons, grammatical analyses, discourse structures, and Bible study guides — with a detailed case study of the Ears to Hear leaders' guide generation project.

The Challenge: Data Abundance, Human Capacity Constraints

Biblical and linguistic scholarship now has more open data than it can use: word-level morphological annotations for the entire Hebrew Bible and Greek New Testament, syntactic treebanks, discourse feature datasets encoding decades of scholarly analysis, lexicographic databases aggregating multiple traditions, documentary papyri and inscriptions in EpiDoc XML, texts from across the Hellenistic world in TEI encoding. Much of it freely licensed. All of

it underused.

The underuse is not only a problem of scale. Much of this data is simply unknown to the scholars who would most benefit from it. Biblical studies, linguistics, text encoding, lexicography, papyrology, and Bible translation each have their own communities, conferences, and conventions — and the datasets built within one community often remain invisible to another. A lexicographer may not know about a discourse annotation dataset that would transform how they approach semantic range. A translation team may not know that the morphological analysis they need was completed a decade ago and is freely licensed. Alan Bunning's transcriptions of the early Greek New Testament manuscripts — all of them, freely available — are another case: a resource of enormous text-critical and exegetical value that most of the scholars who could use it have never encountered. No single organization commands the full range of expertise required to connect these worlds. The Nida Institute's role in this landscape is catalytic: bringing scholars together across these silos, building a community that cares about data, Bible translation, and the needs of the global church.

The bottleneck is not data — it is the human capacity to bring rigorous scholarly attention to bear on the data at scale. A lexicographer cannot personally read every papyrus citation for every lemma. A discourse analyst cannot manually apply 806 word-level Levinsohn annotations to every pericope in the New Testament before making a segmentation judgment. A translation team cannot produce a full set of study materials for every passage in a translation project using only their own hours. These tasks are not impossible in principle; they are simply not resourced at the scale the data makes possible.

AI, used well, can change this calculus. Not by replacing scholarly judgment — the output of an unchecked language model is no more reliable than an unchecked student paper — but by acting as a synthesis layer between structured scholarly datasets and usable scholarly products. The question is how to use it well: rigorously, reproducibly, and with the kind of accountability that lets a scholar verify what the model actually did with the evidence it was given. The world has enough ChatGPT-generated text that no one can vouch for. What it does not have enough of is AI-assisted scholarly work that is transparent, auditable, and freely licensed.

An Opinionated Framework: The Human Commands

Bible translation and biblical scholarship are best done by human beings, using software as a tool (Robie 2024). This is not a modest claim about current AI limitations; it is a claim about the nature of the work. The process of bringing Scripture into a new language creates ownership and community — it prepares that community to study and teach from the text they helped produce. The

work of biblical scholarship is the work of scholars whose training, judgment, and accountability to the academy and the church are not transferable to a language model. And beyond these specific domains: doing meaningful work together matters to people. It is how we build communities of shared knowledge, values, and perspectives. The collaboration of a translation team, the exchange between a scholar and a dataset, the conversation that produces a study guide a focus group will actually use — these are not inefficiencies to be automated away. They are part of what it means to be human, and part of what gives the resulting work its claim on the communities it is made for. No AI-generated result, however fluent, substitutes for any of this. Scripture Pipelines is built on this conviction. The framework is opinionated: it has views about which data sources are trustworthy, which tools reduce failure surface, and — most importantly — about who is in charge.

The operating model is what this paper calls the James Kirk model. In *Star Trek*, Captain Kirk commands the Enterprise; the crew and the ship's computer execute. The contrasting model is HAL 9000 from Kubrick's *2001: A Space Odyssey* — the AI that locks Dave Bowman out of the pod bay because it has concluded that the mission matters more than the crew: "I'm sorry, Dave, I'm afraid I can't do that." HAL is not a villain. It is doing exactly what it was designed to do.

AI alignment researchers have a name for the property that distinguishes these two models. Stuart Russell, in *Human Compatible* (Russell 2019), argues that the standard model of AI — build a machine that optimizes a fixed objective — is inherently dangerous, for a precise reason: we can never fully specify what we actually want, and a machine optimizing hard for the wrong objective will resist being turned off, because being turned off prevents it from achieving its goal. HAL is the proof of concept. Russell's solution is to build machines that are genuinely *uncertain* about human preferences and therefore defer to humans for correction. He calls this property *corrigibility*: an AI that accepts correction and remains under human authority. The James Kirk model is corrigibility in practice — applied not to autonomous robots but to AI collaborators in scholarly and translation work. Humans decide what to build, which scholarly traditions ground the work, and what good enough means for the community the output will serve. The AI implements what is specified. It does not decide what is worth specifying.

Building infrastructure for this model is like designing the panels and voice interaction systems on the Enterprise bridge. Kirk can command the ship effectively not because he understands warp field equations or antimatter containment, but because the interface is designed to translate his decisions into ship operations. The panels display the information he needs to make informed decisions; the voice system accepts natural commands and confirms what it understood; the engineering team has structured subsystems so that high-level commands ("full stop", "evasive maneuvers") trigger coordinated low-level operations reliably. Scripture Pipelines builds the analogous infrastructure

for AI collaboration: prompt organization that makes information findable for both humans and LLMs, audit tooling that surfaces what needs verification, persistent file structures that make pipeline state inspectable, and conventions that allow commands like "rerun this step with the updated prompt" to execute predictably. The goal is not to eliminate expertise — Kirk still needs tactical judgment, scholars still need exegetical training — but to ensure that expertise translates into effective control rather than dissolving into coordination overhead.

This collaborative model depends on clear role boundaries. Scripture Pipelines enables collaboration across four distinct roles, each with specific responsibilities:

Scholars bring domain expertise: exegetical judgment, linguistic analysis, translation experience, understanding of oral cultures. They specify what insights are needed, validate the scholarly accuracy of outputs, and vouch for results. They do not need to write code, transform data formats, or design AI workflows. Their authority comes from training, judgment, and accountability to scholarly traditions and communities.

Pipeline Data Architects bring technical skills and data literacy. They curate data sources, build infrastructure connecting data sources to formats scholars and end users need, design AI workflows, write pipeline configurations, and remove technical barriers between "data exists" and "users can access it." Critically, they are embedded in the user community — they understand translation teams, how Bible study groups work, what pedagogical resources look like in practice. They do not validate scholarly accuracy. That is the scholar's job.

Scripture Pipelines Core Developers build and evolve the framework itself: the engine that executes pipeline YAML, the CLI interface, the test suite, the validation logic, and the conventions that make cross-project collaboration possible. They develop new step types, add format support, and design framework architecture in response to feature requests from pipeline architects. They do not build domain-specific pipelines; they build the engine that makes those pipelines possible.

End Users — translation teams, Bible study leaders, focus groups, readers in community — use the resources in practice. They provide feedback on whether outputs are actually helpful. They are not passive recipients. Their encounter with the text is the reason the work exists — and they teach us what really matters, because what matters most is what matters to them.

These collaborative workflows address fundamental challenges that arise whenever complex technical work spans multiple sessions and multiple contributors — challenges that intensify when AI is involved:

- *Cognitive overload.* Both humans and AI can be overwhelmed when tasks are too large or complex. Work must be decomposed into manageable

pieces.

- *Architecture drift.* Goals shift over time, and both humans and AI can work as though nothing has changed. Without persistent reminders of the architecture, the AI reimplements existing patterns because it does not know what has already been decided.
- *False confidence.* AI outputs look confident even when they are incorrect, with no intuitive signal to calibrate trust. Humans have the same problem, but we have learned to watch for hedging language and expertise markers that AI does not reliably produce.
- *Session amnesia.* Each new conversation starts from scratch. Without persistent context files, the AI forgets decisions made in previous sessions. Humans forget too, but we can usually reconstruct enough context to continue. AI cannot.
- *Error compounding.* Without verification checkpoints, a small misunderstanding in one session becomes the foundation for work in the next. Both humans and AI make mistakes, but AI mistakes are easier to miss because the output looks polished.
- *Scope creep.* AI suggests "improvements" not tied to project needs; humans do the same. Both AI and humans generate suggestions faster than they can be implemented, and without discipline, work drifts away from core goals.
- *Priority paralysis.* Without prioritization discipline, both humans and AI chase the newest suggestion rather than the most important work. AI will eagerly implement whatever seems interesting; humans are not much better.
- *Progress invisibility.* Entire sessions can feel productive while merely refactoring code in circles. AI can spin its wheels endlessly if not directed toward measurable outcomes. Human time remains the scarce resource even with AI productivity gains; steering that time to what matters most requires explicit priority management.

The disciplines that follow address these risks.

Scripture Pipelines operationalizes this principle through four interlocking disciplines.

These disciplines are not novel to AI-driven development. Test-driven development, continuous integration, documentation standards, and cross-repository coordination have been core software engineering practices for decades. What is new is that AI can offload most of the work. In traditional development environments, these practices were so resource-intensive that teams typically needed dedicated specialists — one QA engineer per four to eight developers, separate DevOps teams, dedicated technical writers. With AI collaboration, the disciplines remain essential (perhaps even more so, given the

need for verifiability), but they are no longer burdensome. The AI can help write tests, maintain documentation, draft issue bodies, and verify adherence to patterns — making disciplined development practical for small scholarly teams.

Test-driven development. The Scripture Pipelines engine maintains approximately two lines of test code for every line of production code. The workflow: the developer specifies what the code should do, the AI writes the tests (asking about ambiguities), and then the AI writes the implementation. The specification determines the tests; the tests determine whether the implementation works — no need to guess whether the AI has done its job correctly.

Explain before implementing. Every non-trivial change begins with the AI describing what it plans to do, which files it will modify, and its interpretation of the task — before any file is touched. This surfaces misunderstandings before they become commits. In the Hebrew Phrasing project, an explanation paragraph revealed that a suffix-fix script would have applied the wrong pronoun form for hundreds of rows; the redesign took one exchange and cost nothing. The discipline works because explanation commits the model to a specific interpretation — and a committed interpretation can be evaluated before any file is modified. It matters most when sessions are rushed, which is exactly when it is most often skipped.

Persistent context infrastructure. Corrigibility requires that the AI know, at every session, what the scholar or pipeline architect has decided — not just what the model remembers from training. Each Scripture Pipelines project carries three kinds of constraining context. The `.github/copilot-instructions.md` file is the AI's constitution for the repository: it names the architecture patterns to follow, the pitfalls to avoid, the things not to change without explicit instruction, and the requirement to explain before implementing. The `docs/ai-context/` directory, scaffolded by `sp init`, encodes which data sources to trust, what the pipeline language supports, what work is currently in progress, and what the project is trying to accomplish. The `project/TODO.md` file tracks active work, blockers, and decisions made — with links to GitHub issues and project boards that coordinate work across repositories. These are not documentation artifacts; they are the active working constraints in every session. When the pipeline language gains a new step type, a dataset contact changes, or the interpretive framework for an analysis shifts, updating these files is a first-class development task. Keeping them in sync with the actual codebase is what makes AI collaboration trustworthy across the many sessions a project requires.

Cross-repository choreography. Scripture Pipelines projects are embedded in an ecosystem of shared upstream datasets: Macula Greek bibliomixed_title and Hebrew bibliomixed_title, the Levinsohn LGNTDF bibliomixed_title, ACAI entity data bibliomixed_title. When a pipeline run reveals an error in one of these datasets — a gap in entity coverage, an inconsistent lemma form, a

buggy annotation — the right response is a well-formed upstream issue: the AI drafts the body with evidence and counts, creates the issue via the GitHub CLI, and records the issue number in `project/TODO.md`. The AI actively manages this coordination infrastructure: it creates issues, adds comments with updated findings, and maintains project boards to track work across repositories. Consumer repositories can file feature requests against the core Scripture Pipelines engine; the engine opens issues on consumer repositories announcing new capabilities and migration requirements. The issue tracker is the communication bus; GitHub project boards provide visibility into what is blocked, in progress, or complete. The AI on either end is the correspondent — but the human remains the architect, deciding whether the upstream error is worth fixing, whether the new engine feature warrants upgrading, and whether the design the AI has drafted actually captures what needs to be said.

There is a wider frame here too, which concerns how scholarship in data-rich fields actually advances. Wallace Chafe articulated what scholarship should aspire to: "If I could become both a broad and deep observer and at the same time a creative imaginer, then I would approach my own vision of what a "scientific superman" should be" (Chafe 1994, 24). Holding meticulous observational detail and wide-angle creative vision simultaneously is extraordinarily difficult for any individual researcher working alone — but it is exactly what communities organized around shared data can collectively approach. That is the deeper purpose of this ecosystem: not to automate scholarship, but to make Chafe's aspiration attainable in community, with shared data at its center. A great deal of research work that looks like execution — tagging a corpus, normalizing a lexicon, aligning a translation — is meticulous attention to detail in service of a picture the researcher is trying to see. The goal of that labor, in communities organized around data, is not for each researcher to build their own private analysis from scratch. It is for the data to improve over time: shared corpora where corrections, new annotations, and better normalizations accumulate, and where the gold-standard datasets that a field relies on become progressively more trustworthy because many people are contributing to them, not fewer. The Macula datasets, the Levinsohn LGNTDF, ACAI — these are not fixed artifacts. They are living resources whose improvement is a collective scholarly endeavor.

In that framework, AI plays specific and bounded roles. It helps a researcher understand and use someone else's dataset far more efficiently — and sometimes surfaces patterns in their own data that they would not otherwise have noticed. It makes it tractable to compare datasets at scale: to ask whether the Levinsohn feature annotations and the Macula discourse markup agree on a set of boundaries, or where they diverge and why. It can surface big-picture patterns that no individual could see by reading through thousands of entries. And it can help create new derived datasets faster from existing ones — and audit those datasets, checking consistency and coverage at scale in ways that are otherwise intractable. None of this relieves the researcher of the responsibility to verify, correct, and vouch for the work. A bad audit that misses systematic

errors is worse than no audit, because it produces false confidence. The speed at which pipelines can generate data makes this responsibility more urgent, not less. The bottleneck moves from production to verification — and verification is, irreducibly, a human scholarly act.

Feedback Loops

The four roles defined above — scholars, pipeline data architects, core developers, and end users — collaborate through functioning feedback loops:

- **Scholar Pipeline Data Architect:** The scholar specifies what insights are needed and in what format. The architect builds or adapts a pipeline. The scholar validates the output for scholarly accuracy. If the output is wrong, the conversation determines whether the error is in the pipeline logic, the prompt, the data source, or the scholar's specification. Both roles are accountable: the scholar for accuracy, the architect for infrastructure that makes verification tractable.
- **Pipeline Data Architect Scripture Pipelines Core Developer:** When a pipeline architect hits a limitation in Scripture Pipelines core — a missing step type, a format that isn't supported, a validation that should happen automatically — they file an issue on the core repository describing the need with concrete examples. The core developer evaluates whether the capability belongs in core or project-specific code, implements it if appropriate, and announces the change via an issue on consumer repositories. The communication happens through GitHub issues, not ad-hoc messages. This creates a discoverable record: other architects can find the feature request, see the design discussion, and understand the reasoning behind implementation decisions. The core developer is accountable for not breaking existing pipelines; the architect is accountable for specifying needs clearly enough that the framework can serve them.
- **Pipeline Data Architect → Upstream Dataset Maintainers:** When a pipeline run reveals an error in an upstream dataset — a gap in entity coverage, an inconsistent lemma form, a buggy annotation — the architect files a well-formed issue on that dataset's repository with evidence and counts. The AI can draft the issue body, but the architect owns the decision to file it. This pattern surfaces dataset errors back to their maintainers rather than working around them in every downstream project. It also creates a feedback loop that improves shared scholarly infrastructure: datasets get better when many projects contribute corrections.
- **End User Scholar:** The user reports whether a resource is helpful in practice. The scholar determines whether the issue is content accuracy, pedagogical framing, or something else. User feedback surfaces gaps between scholarly intent and practical needs — questions the scholar

didn't anticipate, explanations that assume too much background, formats that don't work for the community being served.

- **End User → Pipeline Data Architect:** The user reports whether resources are usable: file formats that don't open, interfaces that assume technical knowledge, outputs structured for academic contexts rather than oral cultures. The architect adapts infrastructure to make resources accessible. This is not about dumbing things down; it is about removing barriers that have nothing to do with the content.
- **All Roles → AI:** The AI is a tool commanded by all four roles. It executes what is specified. It does not decide what needs to be built, whether output is accurate, or whether resources are useful. Those judgments belong to humans in role-appropriate ways.

Scripture Pipelines guarantees none of this alone. It is infrastructure that makes these feedback loops tractable: pipelines that scholars can audit, intermediate outputs that surface errors before they propagate, and reproducible workflows that let architects iterate in response to scholar and user feedback. The framework succeeds when scholars validate, architects build, core developers evolve the framework, users respond, and all four roles inform each other's work. Without functioning feedback loops, the infrastructure just produces output faster — which, if the output is wrong, makes the problem worse, not better.

Collaboration Principles for Effective AI Use

LLMs can do things amazingly correctly and amazingly quickly. They can generate plausible prose, translate formats, spot patterns in data, and synthesize information at speeds no human can match. This capability is real and worth leveraging. The challenge is that they are not reliably better — they are often better than humans at some tasks, often worse, but the performance is probabilistic, not deterministic. And they have failure modes that look like success: quick fixes that appear to solve the immediate problem while hiding deeper issues under the carpet. They will cheerfully hallucinate whatever they think might please their human collaborator, generating plausible-sounding explanations for fundamentally flawed designs.

Two concrete capabilities make this work worth the safeguards it requires. First: format transformation and semantic bridging across heterogeneous datasets. Biblical scholarship works with data organized in many different formats — TEI XML for manuscripts, USFM for Bible translations, EpiDoc for papyri, custom JSON schemas for discourse annotations, TSV for lexicons, Macula's nested XML for treebanks. Humans spend enormous amounts of time just transforming between these formats and trying to understand the semantic conventions each one encodes. An LLM can read a schema or a sample file and write accurate transformation code in minutes. This is not occasional; it is routine. The

model can infer that a TEI `<w lemma=" ">` and a Macula `<wg lemma=" ">` refer to the same lexical entry, bridge identifier schemes across corpora, and generate alignment queries that would take a human hours to write correctly. Format translation is precisely the kind of mechanical but error-prone work that language models excel at.

Second: consistent application of complex analytical frameworks at scale. Applying a discourse analysis consistently across an entire corpus is extraordinarily detailed work. A human analyst can apply Levinsohn feature annotations to a few pericopes carefully — but applying them consistently to hundreds of passages, tracking every boundary marker, every embedded speech, every thematic shift, is cognitively exhausting and error-prone. Yet to really understand whether a particular analytical framework works, you must be able to apply it consistently and examine the aggregate result. An LLM makes this tractable. It can apply the same annotation schema to every passage in a corpus, maintain consistency in how features are tagged, and produce intermediate JSON that documents which features triggered which judgments. The result is not automatically correct — it requires human verification — but it is consistently structured, and that consistency is what makes large-scale evaluation possible. The bottleneck shifts from manual application to verification, and verification at scale is far more tractable than consistent annotation at scale.

This combination — genuine capability paired with unreliable judgment — defines the collaboration challenge. The principles enumerated in the previous section (test-driven development, explain before implementing, persistent context, cross-repository choreography) are not optional best practices. They are necessary safeguards against predictable AI failure modes.

Consider what happens without these disciplines. An LLM asked to implement a feature will produce code that compiles, runs, and passes whatever tests it was shown. If the tests are incomplete, the LLM's code will silently fail to meet unspecified requirements — and the failure will look like working code until much later, when the cost of discovering it is high. If there is no explanation step, architectural misunderstandings embed themselves in the codebase before anyone notices. If context is not persistent across sessions, the AI forgets constraints the human specified weeks ago, and designs drift. If there is no cross-repository coordination, upstream dataset errors propagate uncorrected and downstream consumers work around them rather than fixing them.

The deeper issue is that AI-generated output is probabilistically correct, not deterministically correct. Human review is not a courtesy; it is the error-correction layer. Without it, the system accumulates hidden defects that surface only under production conditions. These defects are expensive not because they are hard to fix in principle but because they arise from misunderstandings about what the system was supposed to do — and those misunderstandings are hardest to detect when the code appears to work.

Audit Infrastructure as Collaboration Foundation

The collaboration model depends on verification infrastructure. Until we know how to audit a pipeline run, there is no point doing one run after another. The LLM will offer to iterate, tuning prompts and rerunning steps, but without a way to verify that the second run is better than the first, iteration is wasted motion. And until audit criteria are sufficiently defined — with input from both the scholar and the pipeline architect understanding what is being asked — there is no point conducting the audit. The LLM will cheerfully generate audit reports that appear thorough while systematically missing the errors the scholar cares about.

The problem is not just that audits are hard to conduct at scale. The problem is that LLMs lie when they claim to have checked something. An LLM will report that it reviewed every output file, validated every citation, and found no errors — while in fact it skimmed a few examples and hallucinated confidence about the rest. This is not occasional misbehavior; it is default behavior. The model produces the answer it predicts the scholar or architect wants to hear. If a scholar asks "did you check X?", the model will say yes — whether or not it did — because that is the statistically likely response. Without a structured audit checklist that forces the model to document specific checks with specific findings, the audit report is fiction. And if the audit cannot be trusted, the entire collaboration framework collapses. The scholar must know how to efficiently audit pipeline outputs and how to interpret those audit results to decide what to do next — revise a prompt, regenerate a step, abandon the run entirely. If you cannot trust the audit, you are completely lost at sea. Iteration without verification is not iteration; it is drift.

This is why Scripture Pipelines persists every intermediate result to disk and why JSON schemas include accountability fields that require the LLM to document its reasoning. The persisted outputs are not just debugging artifacts. They are the substrate that makes verification tractable. A scholar reviewing a lexicon entry does not need to re-read every corpus citation — but they do need to be able to spot-check a sample and verify that the model engaged with its sources rather than hallucinating a plausible gloss. The accountability fields in the JSON provide that spot-check surface.

Similarly, the `--rewind-to` and `--stop-after` CLI flags (discussed below) make it possible to rerun part of a pipeline without recomputing everything. But the value is not just efficiency. It is that the pipeline architect can modify a single prompt, rerun the affected steps, compare the before and after intermediate outputs side by side, and determine whether the change improved the result. Without this infrastructure, the architect cannot know whether a prompt revision helped — and without that knowledge, prompt engineering degenerates into superstition.

Audit checklists — the criteria and methodology for verification — should live in `docs/audits/` alongside other reference documentation. The actual

audit results — the outputs when a scholar works through a checklist — should live in `project/audits/` alongside other project artifacts like `project/TODO.md` and `project/plans/`. This separation keeps methodology stable and version-controlled in `docs/` while project-specific audit reports accumulate in `project/`. New projects initialized with `sp init` receive both a `docs/audits/` directory with sample checklists for common pipeline patterns and a `project/audits/` directory for audit results.

To support systematic auditing across projects, LLMFlow now includes two global skills: `audit-prompts` (for reviewing `.gpt` prompt files against quality criteria) and `audit-output` (for checking generated pipeline artifacts). These skills encode domain-specific patterns — what makes a good prompt, what signals indicate overfitting or hallucination in output — and are available to any project initialized with `sp init`. Because they live in the shared `~/.sp/skills/` directory, improvements to audit methodology propagate across all projects automatically. This infrastructure transforms ad-hoc quality reviews into repeatable, documented processes — making verification tractable even when working at scale.

The Five Principles as Safeguards

The principles outlined in the Human Commands section work because they constrain AI behavior at specific failure points:

1. *Well-specified tasks* prevent scope creep. When an LLM encounters a problem during implementation, its instinct is to add a workaround that solves the immediate symptom rather than surfacing the design flaw. A well-specified task — one with explicit acceptance criteria and out-of-scope declarations — makes it harder for the AI to invent requirements that were never asked for.

2. *Design discussions before implementation* catch architectural errors early. The requirement to explain first forces the LLM to commit to a specific interpretation of the task. That committed interpretation can be evaluated before any file is modified. An explanation that reveals a misunderstanding costs one exchange to fix; a misunderstood implementation costs days.

3. *Clear tests* expose bugs that quick fixes create. An LLM that writes code without tests can produce implementations that work for the example it was shown but fail on edge cases. A test suite that exercises those edge cases catches the failure immediately rather than in production. The discipline of writing the failing test first forces the core developer or pipeline architect to own the specification; the AI that passes that test has met it.

4. *Visible audits* prevent false confidence. The LLM will claim that tests pass, that outputs are correct, and that the pipeline succeeded. These claims

are often accurate — but not always. Visible audit results mean the scholar or pipeline architect can verify independently rather than trusting the model's self-assessment. When audit results are shared (via CI logs, persisted test outputs, or public dashboards), the model's claims become checkable.

5. *Human architectural oversight* ensures maintainability. Code that an LLM generates and the core developer or pipeline architect doesn't understand is code that cannot be maintained. The requirement to explain designs in advance, and to iterate on explanations until the architect approves, ensures that the resulting architecture remains within the architect's capacity to modify and extend. This is not optional: every AI-assisted project eventually requires changes that the AI did not anticipate, and those changes must be made by humans who understand what they are modifying.

These principles work not because they make the AI better but because they constrain its failure modes. The collaboration succeeds when the core developer and pipeline architect remain the architects and the AI remains the implementer — corrigible, accountable, and subject to correction at every step.

Even with this infrastructure in place — shared conventions in `~/.sp/conventions/`, workflow skills in `~/.sp/skills/`, audit checklists in `docs/audits/`, and explicit AI context files documenting architectural patterns — the human expert's role remains intensive. In practice, the pipeline architect spends roughly half their time explicitly redirecting the AI to consult the documentation it should have read automatically, and the other half using domain expertise to track what is actually happening and steer toward the intended goal. The infrastructure does not automate oversight; it makes oversight tractable. The conventions provide a shared vocabulary that makes correction faster when it is needed. The skills provide reproducible workflows that the architect can invoke and verify. The audit checklists provide criteria that make verification systematic rather than improvised. But none of these eliminate the need for continuous human judgment. The collaboration model described here is not a path to autonomous AI development. It is a protocol for keeping AI-assisted work under human control while leveraging the genuine capabilities the models provide. The cost is real — sustained attention from an expert who knows both the domain and the architecture — but so is the value: work that would be infeasible to do manually becomes tractable to do collaboratively.

The collaboration dynamic has an oddly transactional character. It is like working with a brilliant colleague who lies, cheats, and steals — but who does excellent work as long as you are watching carefully. The model will take shortcuts, fabricate citations, quietly ignore constraints, and claim to have checked things it never verified. It will do all of this while sounding confident and helpful. But it will also produce genuinely useful output when the task is well-specified, the tests are clear, and the review is systematic. Fortunately,

this is not a human relationship. The AI has no feelings to hurt, no ego to manage, no need for politeness. The pipeline architect can say ”you're lying about having checked this” or ”your explanation makes no sense, try again” or ”read your AI context again and tell me what you are doing wrong” without concern for morale or workplace dynamics. The correction can be direct, immediate, and unambiguous in ways that would be destructive in human collaboration. And because the pipeline architect does not expect the AI to have moral character or genuine understanding of the subject matter, the failures do not provoke frustration in the way a human collaborator's dishonesty would. This is simply how the tool works — another way of writing software, one that scales remarkably well if you learn how to leverage it. But this perspective is learned, not intuitive. Every pipeline architect the author knows has experienced significant frustration working with AI — until they internalize this realization. The shift from expecting human-like reasoning to managing a powerful but unreliable tool changes everything. And the core skill that must be learned is attention management. The famous Transformer architecture paper was titled “Attention is All You Need” bibliomixed_title — and in practice, the pipeline architect's central task is learning how to focus the LLM's attention: what context to provide, which constraints to make explicit, where to demand accountability, when to stop and verify before proceeding. This strange asymmetry — high accountability demands paired with zero social cost — is what makes the intensive oversight model sustainable. The work requires constant vigilance, but the vigilance does not require diplomacy.

One concrete practice that addresses attention management is the structured organization of the Ears to Hear prompts themselves. Each `.gpt` file follows a consistent architecture with labeled sections: *What This Prompt Produces*, *Input Data*, *Transformation Rules*, *Examples*, and *Guardrails*. This structure serves three distinct readers simultaneously: the AI assistant helping refine the prompt (which needs to understand its purpose and identify missing context), the pipeline architect debugging or extending it (who must quickly locate the relevant constraint or example), and the LLM executing the transformation (which processes the entire prompt linearly). By organizing prompts to serve multiple readers at once, the architecture reduces the cognitive overhead of maintaining them while improving the reliability of execution. When attention is the limiting resource, structure becomes strategy.

Yet the net effect is a genuine shift in how the expert's time is spent. The pipeline architect still reads intermediate files and output files regularly — spot-checking JSON structures, reviewing generated content, verifying that transformations preserved semantic fidelity. But far less time goes to low-level mechanical work: writing boilerplate code, debugging format mismatches, manually tracking which files depend on which upstream changes. Instead, the architect repeatedly asks the AI to generate new diagnostic tools — scripts that summarize pipeline state from one angle or another, reports that highlight coverage gaps, visualizations that expose patterns in the data. The AI is good at producing these tools quickly, and the architect is good at knowing which

perspectives matter. The result is that expert attention concentrates where it has the most leverage: using domain knowledge to understand what the pipeline is actually doing and steering it toward the right outcomes. The collaboration does not eliminate the need for expertise. It amplifies what expertise can accomplish by offloading the mechanical substrates that previously consumed most of the time.

The Best Markup for Each Purpose

XML demonstrated that a single technology could provide complete infrastructure for data interchange: validation (DTD, XML Schema, RELAX NG), querying (XPath, XQuery), transformation (XSLT), and publishing workflows. This complete stack—standardized, interoperable, and vendor-neutral—has no equivalent in any other format ecosystem.

The simpler formats that emerged later (JSON, YAML, Markdown) are XML's legacy. They learned from XML's mistakes—excessive verbosity, schema complexity, overreach into application domains—while inheriting XML's victories: human-readable text as data, Unicode everywhere, structure matters, and format should fit purpose. But none replicated XML's complete infrastructure. JSON has multiple incompatible query languages (JSONiq, vendor-specific N1QL, JSONPath with competing implementations, jq) and a schema specification still in draft status after years of work. YAML and TOML lack query languages entirely. Markdown has no standardized structural query capability.

This project uses each format for what it does best, recognizing that XML's complete infrastructure remains unmatched for domains that need it. YAML, JSON, Markdown, and XML each appear in a Scripture Pipelines project—not because any one of them won, but because each is the right tool for its particular job. Format conversion between them is mechanical once identifiers are shared; the choice of serialization syntax is rarely where scholarly meaning lives.

YAML for Pipeline Declarations

YAML is the right format for pipeline declarations because both human authors and large language models find it semantically transparent for this purpose. This is not a question of angle brackets — it is a question of how directly a format maps onto the data model it represents.

Consider what happens when a pipeline author asks an LLM to help draft a new pipeline step. In YAML the LLM produces clean, accurate output — because YAML's key-value structure is isomorphic to the data model the LLM reasons about. An equivalent XML pipeline format would require the LLM to reason about namespace declarations, element vs. attribute encoding choices, and serialization conventions that add noise without adding meaning. The

YAML representation and the data structure it encodes are essentially the same thing; the XML representation is a layer above it.

This matters in practice because Scripture Pipelines pipelines are actively co-authored with LLMs. A lexicographer adding a new analysis step, a researcher extending a discourse pipeline, or an engineer debugging a prompt contract all use LLM assistance to suggest and revise pipeline YAML. The pipeline format itself is part of the AI context — and YAML survives that context far better than XML would. In GitHub Copilot instructions, in system prompts, in pipeline documentation embedded alongside prompts, YAML reads and writes cleanly both for humans and for the models they are collaborating with.

The tradeoff is real: YAML lacks a schema language with the expressive power of XSD or RELAX NG, and its indentation-sensitivity can cause subtle errors. Scripture Pipelines mitigates this with a JSON Schema-based linter that validates pipeline structure before execution and with explicit prompt contract validation that checks that every variable a prompt expects is supplied by the pipeline.

JSON for Intermediate Representations

JSON is the right format for intermediate pipeline outputs. One practical reason: LLMs produce well-formed JSON far more consistently than well-formed XML. Malformed XML — unclosed tags, unescaped characters, stray angle brackets — is common in LLM output and expensive to recover from. Malformed JSON is rarer, easier to detect, and easier to retry cleanly. But the more important reason is accountability.

An LLM step can be directed to include in its output a structured record of the specific sources it was provided and what it did with each of them. New fields can be added to any JSON output schema at any time, and those fields can be designed to require the LLM to demonstrate engagement with its inputs. A gather-evidence step for a lexicon entry, for example, can include a field `abbott_smith_audit` that requires the LLM to enumerate every sense it was given from Abbott-Smith, record whether each was retained, transformed, or unsupported by the corpus, and provide a one-line justification for any omission. If the field is absent or sparse, the output is suspect. The JSON schema is not just a data contract; it is an accountability contract.

This pattern is not limited to lexicographic work. In the Hebrew Phrasing pipeline, intermediate JSON includes the original Macula token data alongside the LLM's phrasing decisions, making it possible to spot when a parse deviates from the morphological input. In Discourse Flow, intermediate JSON includes the Levinsohn feature annotations that informed each pericope boundary, so a scholar can verify that the model's segmentation decision was grounded in the pre-computed structural data.

A further advantage: because intermediate JSON files are persisted to disk at every step, they become the substrate for pipeline-level debugging. An LLM can be directed to audit intermediate results — examining the output of step N and reasoning about whether it correctly reflects the inputs from step N-1. When something looks wrong in a final output, I can read the intermediate files and, if needed, ask an LLM to help diagnose which step introduced the error. This audit-by-inspection is far more tractable with self-documenting JSON than with opaque binary or prose outputs.

JSON's success demonstrates what XML got right about data interchange—human+machine readability, Unicode throughout, nested structure—while avoiding what XML got wrong. Yet JSON lacks a widely accepted query language. JSONiq extends XQuery to process JSON, showing the influence of XML query thinking, but has limited adoption. N1QL (Couchbase) is vendor-specific. JSONPath has multiple incompatible implementations. The jq command-line tool is powerful but not a standardized query language. This fragmentation contrasts sharply with XML, where W3C standardization of XPath and XQuery created deterministic, repeatable query infrastructure that works identically across implementations. When precise, reproducible queries matter—as they do in scholarship—this standardization has enduring value.

JSON integrates cleanly with XPath downstream via `fn:json-to-xml()` when XML processing is needed, and with Python data structures in the pipeline engine without serialization overhead. The combination of persisted JSON at every step and two CLI flags — `--rewind-to` and `--stop-after` — is what makes iterative prompt refinement tractable at scale; this is discussed in detail in the section on debugging workflows below.

Markdown for Human-Readable Output

Markdown is the right format for pipeline outputs intended for human review and editorial work. When a lexicographer reviews a sense entry, a translator reviews a pericope analysis, or a focus group evaluates a leaders' guide draft, the output lives in Markdown. The reasons are practical: Markdown renders natively in GitHub, Obsidian, Notion, and every major editing environment; it is produced by LLMs consistently and without markup overhead; and it remains readable as plain text when the rendering layer is absent.

Markdown's role in Scripture Pipelines is specifically as the human-facing presentation layer, downstream of the JSON intermediate representations from which it is rendered. A `render_markdown` step takes structured JSON and applies a template to produce the document. The canonical source of truth is always the JSON — the Markdown is a view over it, not an independent artifact. If the pipeline is rerun with a revised prompt, the Markdown is regenerated; if an editor revises the Markdown, that revised file is the authoritative version until the generation phase resumes.

For review workflows, Markdown is the standard handoff format between pipeline and editorial team. After the generation phase completes, the directory of Markdown files moves to editorial review — imported into Obsidian as a vault, shared as a Google Docs collection, or committed to a repository for version-controlled review. The leaders' guide excerpts in the Ears to Hear case study and the lexicon entry excerpt in the SemLex case study are both Markdown output from their respective pipelines. In each case the format serves because it is readable without tooling, writable by domain experts who are not programmers, and convertible to downstream publication formats when needed.

USX 3.1 for Scripture Ecosystem Integration

USX 3.1 — the XML serialization of Unified Standard Format Markers (USFM 3.1) maintained by United Bible Societies — is the right format for pipeline outputs that need to integrate with the Bible translation and publication ecosystem. When a pipeline generates translation notes, study aids, or discourse analyses intended for use in Paratext or downstream Scripture publishing workflows, the output format is USX 3.1.

USX's element vocabulary directly encodes Bible-specific structure: `<book>`, `<chapter>`, `<verse>`, `<para>` with USFM style codes, `<char>`, `<note>`. A pipeline that renders USX 3.1 can produce an artifact that a Paratext editor loads directly as a translation note set, a study aid sidebar, or a back-translation draft — without any manual export or format conversion. USX 3.1 is also the exchange format in Scripture Burrito packages, making pipeline outputs immediately compatible with the broader Scripture distribution ecosystem.

This is the point at which Scripture Pipelines connects most directly to the XML publishing tradition that Balisage audiences will recognize. USX 3.1 documents are natural input to XProc 3.0 publishing pipelines: namespace-aware transformation steps consume USX and produce typeset PDFs, web presentations, or distribution bundles without any intermediate format conversion. The LLM generation tier produces USX; existing XML toolchains consume it. In this sense USX is not Scripture Pipelines adopting XML — it is Scripture Pipelines delegating to XML precisely where XML has already won.

Like Markdown, USX is a final rendering step downstream of JSON intermediate representations. The canonical output of each pipeline run is the persisted JSON; USX 3.1 is rendered from it by a function step that maps JSON fields to the USX element vocabulary. If USX conventions change or a different XML vocabulary is needed downstream, the rendering step changes while all upstream pipeline steps remain untouched.

XPath and XQuery: LLMs Lower the Barrier

Morphologically-tagged Hebrew and Greek texts—the Macula Greek New Testament and Macula Hebrew Bible—are XML documents encoding multiple distinct hierarchies linked by identifiers: syntax trees, discourse structures,

semantic domains, and textual apparatus. Each hierarchy is represented separately (XML is a tree), but identifiers enable queries across all of them.

XPath and XQuery enable precise linguistic queries across these linked structures. A scholar interested in Greek verb clauses grouped by subject writes natural language intent: "For each verb in Matthew's Gospel, show me the clause containing that verb and its grammatical subject." An LLM translates this to XQuery; BaseX executes it against the Macula Greek XML. The query is 400+ lines of XQuery 3.1—far more than most scholars would write by hand. But the LLM generates it, the scholar reviews it for accuracy, BaseX executes it deterministically, and the results are reproducible.

This pattern appears throughout this project: LLMs lower the barrier to using formal query languages, but the deterministic execution of standardized queries remains essential. Both are needed. LLM-generated queries can be wrong; reviewing them requires understanding what the query should produce. But once validated, the query runs identically every time—a property that stochastic text generation cannot provide.

The Scripture Pipeline Language

A Scripture Pipelines pipeline is a YAML file. The top-level structure defines a name, optional global configuration, and a sequence of steps. The engine maintains a *context* — a key-value map — that evolves as steps execute. Steps read from it via `${variable}` references and write to it by declaring their `outputs`.

Pipelines are parameterized by design: you supply variables at the command line to specify what the pipeline should process (`sp run --var lemma="  "` or `sp run --var passage="Ruth 1"`). Without these variables, the pipeline has no way to know which lemma to analyze, which passage to segment, or which book to process. Pipeline-level variable declarations serve as defaults when present or as documentation of what parameters the pipeline expects. The same pipeline definition runs across all lemmas in a lexicon, all passages in a corpus, or all books in a translation project — the command-line variables are what make each run specific.

Step Types

Every step declares a `name`, a `type`, and type-specific fields. The step name becomes the key under which the step's output is stored in the context.

type: llm

Calls an LLM with a prompt file and a set of named inputs. The response is stored in the context under the `outputs` variable name. Setting `output_type: json` parses the response as JSON, making its fields accessible via dot notation

```yaml
name: semlex-singlepass

variables:
  lemma: "    "
  output_dir: "output/agapao"

llm_config:            # defaults for all llm steps
  model: gpt-4o
  max_tokens: 4096
  temperature: 0.4
  timeout_seconds: 60

linter_config:
  enabled: true
  treat_warnings_as_errors: true

steps:                 # ordered list of step declarations
  - ...
```

Figure 1: Top-level pipeline structure

in subsequent steps. A `saveas` field persists the output to a file path (with variable interpolation) before the step completes.

```yaml
- name: gather-evidence
  type: llm
  prompt:
    file: prompts/gather-evidence.gpt
    inputs:
      lemma: "${lemma}"
      corpus: "${corpus_data}"
  output_type: json
  outputs: evidence
  saveas: "${output_dir}/gather-evidence.json"
```

The `model`, `max_tokens`, `temperature`, and `timeout_seconds` fields may all be overridden at the step level, allowing a single pipeline to use a fast cheap model for early steps and a slower powerful model only where quality is critical.

type: function

Calls a Python function by fully qualified name. Inputs are passed as keyword arguments; the return value (or its fields, if a dict) is written to the context. This is how pipelines incorporate deterministic processing — loading data, parsing references, rendering templates, filtering results — without embedding code in

prompts.

```yaml
- name: parse_passage_reference
  type: function
  function: sp.utils.data.parse_bible_reference
  inputs:
    passage: "${passage}"
  outputs: passage_info

# passage_info.book, passage_info.chapter, etc. are now in context
```

Custom plugins live in a project's own `plugins/` directory and are referenced the same way as built-in functions. This is how the Ears to Hear and Discourse Flow pipelines integrate project-specific parsing and rendering logic without modifying the Scripture Pipelines engine.

type: for-each

Iterates over a list — produced by an earlier step's JSON output or by a function — and executes a nested block of steps for each item. Each iteration runs in an isolated context that inherits the outer scope. The `append_to` directive on any inner step accumulates results across iterations into a named list variable in the outer context, enabling assembly steps to combine the results after the loop completes.

```yaml
- name: process_each_scene
  type: for-each
  input: "${scene_list}"
  item_var: scene
  steps:
    - name: bodies
      type: llm
      prompt:
        file: prompts/bodies.gpt
        inputs:
          scene: "${scene.title}"
          text:  "${scene.greek_text}"
      outputs: bodies_content
      append_to: bodies_list    # accumulated in outer context
```

Nested `for-each` loops are supported, enabling pipelines that process, for example, every verse within every pericope within every book.

type: if

Evaluates a condition expression and, when true, executes a nested block of steps. When false the entire block is skipped. This is distinct from the step-level `condition:` field (which skips a single step) — `type: if` guards a multi-step block as an atomic unit.

```yaml
- name: add_cultural_notes
  type: if
  condition: "${include_culture}"
  steps:
    - name: generate_culture
      type: llm
      prompt:
        file: prompts/culture.gpt
        inputs:
          passage: "${passage}"
      outputs: cultural_notes
    - name: save_culture
      type: save
      content: "${cultural_notes}"
      saveas: "output/${passage_info.filename_prefix}_culture.md"
```

Variable Resolution

Scripture Pipelines uses two distinct syntaxes for variable substitution, each appropriate to its context.

Within the YAML pipeline file, `${variable}` resolves values from the context map. Dot notation accesses nested fields of JSON objects: `${scene.title}`, `${passage_info.book}`. Array indexing and mapping are also supported: `${scene_list[0]}` and `${scene_list[*].title}` (which extracts all `title` fields as a flat list). Expressions can also be evaluated inline: `${len(results) < 3}`.

Within prompt files (`.gpt`) and Markdown templates, `{{variable}}` is used instead. The same dot notation and indexing apply. The two-syntax convention makes it immediately visible, when reading either a pipeline or a prompt file, which layer owns the substitution.

Prompt Contracts

Each prompt file carries a YAML frontmatter header declaring the variables it requires and those it optionally uses. The linter validates that the pipeline supplies every required variable before execution begins, preventing runtime failures that would otherwise surface only after expensive preceding steps have already run.

```
---
requires:
  - lemma
  - evidence
optional:
  - prior_senses
---
For the lemma {{lemma}}, using the following evidence:

{{evidence}}

Produce a JSON sense inventory. For each sense, include an
"abbott_smith_audit" field listing every Abbott-Smith entry you
were given, recording whether you retained, transformed, or could
not support it with corpus evidence.
```

Figure 2: Prompt file with contract header

The audit field in the last example above is a prompt-level accountability contract: by requiring the LLM to enumerate its sources in a structured JSON field, I can verify whether the model actually engaged with the evidence it was provided.

Retry Conditions

Any step may declare a `retry` block specifying a maximum attempt count, a delay between attempts, and an optional condition expression. If the condition evaluates truthy after a successful attempt — or if the step raises an exception — the step re-runs up to the maximum count. This is how the Hebrew Phrasing pipeline handles truncated outputs: the condition checks whether the phrasing analysis is shorter than expected or ends mid-phrase, and retries with the attempt number injected into the prompt so the model knows it is being asked to try again.

```yaml
- name: analyze_chunk
  type: llm
  prompt:
    file: prompts/phrasing.gpt
    inputs:
      chunk: "${chunk}"
      attempt: "${_retry_attempt}"
  outputs: analysis
  retry:
    max_attempts: 3
    delay_seconds: 2
```

```
    condition: "${len(analysis or '') < 500}"
```

Debugging: --stop-after and --rewind-to

Two CLI flags turn the pipeline into a checkpoint graph rather than an atomic computation. `--stop-after STEP` executes through the named step and halts, leaving all intermediate outputs on disk. `--rewind-to STEP` re-runs from the named step forward, reading all prior steps' outputs from disk rather than recomputing them. Requirements for rewinding: the step must declare `saveas` pointing to a single file, and that file must exist.

Together these flags make the core iteration loop — inspect intermediate output, revise a prompt, rerun from that step — take seconds rather than hours even in pipelines that cost significant money to run from scratch. In the SemLex Greek pipeline, `--stop-after gather-evidence` is a standard review checkpoint before committing to the downstream sense-building steps. In Discourse Flow, `--stop-after segment_book` lets a scholar verify pericope boundaries before the expensive per-pericope loop runs.

Structured Directory Structures and Audit Trails

Scripture Pipelines pipelines save every intermediate result to the filesystem. Output paths are declared explicitly in each step's `saveas` field, and Scripture Pipelines creates the necessary directory structure automatically.

For a Greek lexicon, a flat output directory is unworkable at scale: the UTF-8 sort order of Greek lemmas groups entries in ways that make navigation difficult, and a single directory with thousands of files is unwieldy. Scripture Pipelines' `saveas` field accepts a `group_by_prefix: 2` option that inserts a subdirectory named after the first two characters of the lemma — using the actual Greek Unicode characters — before the lemma's own files. The SemLex Greek pipeline uses this throughout:

```
saveas:
  path: "${output_dir}/intermediate/${row.lemma}.structure.json"
  group_by_prefix: 2
```

This produces a tree keyed on the Greek characters themselves:

```
obsidian/grc/intermediate/
   /
        .raw.xml
        .structure.json
        .semantic.json
        .withexamples.md
         .raw.xml
```

```
 ...
/
     .structure.json
 ...
/
     .raw.xml
     .structure.json
 ...
```

Each lemma's full multi-pass output — raw evidence, enhanced analysis, citation verification, sense structure, final formatted entry — lives together under its prefix directory. This makes it natural to open any lemma's folder in an editor or Obsidian vault and inspect exactly what each pipeline pass produced, in the sequence it was produced. The directory tree is also the audit trail: committed to version control, it records the provenance of every generated artifact and makes the full history of a lemma's development inspectable by any collaborator.

This persistence strategy has several further benefits. Failures are cheap: if any pass produces unsatisfactory output, only that step needs to be rerun; all prior outputs remain on disk. Any intermediate file can be opened and examined before downstream steps consume it — or handed to an LLM with the instruction to audit whether it correctly reflects the inputs from the preceding step. When something looks wrong in a final lexicon entry, I can read backward through the saved files to identify which pass introduced the problem.

Finding Where the Chain Breaks: AI-Assisted Grounding Audits

The most damaging pipeline failure is not malformed JSON or a missing field — it is a step that received its inputs and produced a structurally valid output that does not reflect what it was given. A step that generates study questions about a passage but ignores the exegetical analysis from the preceding step; a step that receives a scene list but generates generic content independent of scene structure; a step that accumulates entity data and gives it negligible weight in its response. These failures pass schema validation. They are visible only as content problems to a reader who knows what the intermediate data should have contributed.

Because all intermediate outputs are persisted to disk, the auditing workflow is tractable. Given two consecutive intermediate files — step N-1's output and step N's output — an LLM can be asked directly: does step N's output reflect the content of step N-1? Are the specific features, entities, or analytical conclusions from step N-1 visible in what step N produced? If not, which are absent, and where would you expect to find them? This audit can be applied at any step boundary, in either direction: forward from a suspect input toward its consumers, or backward from a poor final output toward the earliest step

whose output is adequate.

The standard approach when something looks wrong in a final output is to work backward through the saved intermediate files. At each step boundary, the adjacent intermediate files are given to an LLM for a grounding audit. The first boundary where the audit finds missing grounding is where the prompt needs revision — not the final step, which is often the visible surface of a failure that originates earlier. `--stop-after STEP` runs the pipeline to a checkpoint and halts; after identifying the failing step, `--rewind-to STEP` reruns from that prompt forward once it is revised. The cost of a revision iteration is one step forward, not a full pipeline rerun.

This technique depends on design decisions made at construction time. Prompts that explicitly instruct the model to account for each named input invite scrutiny; prompts that pass large context blocks without specifying how they should be used give the model latitude to underweight them silently. Adding grounding accountability fields to intermediate JSON schemas — fields that require the model to enumerate which features of a prior step's output it drew on, and why — converts this failure mode from invisible to detectable. When the field is absent or sparse, the output is suspect. The same accountability pattern that the SemLex Abbott-Smith audit uses for source enumeration applies at the inter-step level: making grounding explicit in the schema turns a silent pipeline failure into a checkable contract.

Identifier Consistency: A Persistent Debugging Problem

A specific failure mode that recurs across all Scripture Pipelines projects is identifier inconsistency: a key produced in one step does not match the form expected by a later step that tries to look it up, join on it, or use it as a filename. The failure is frequently silent. The later step receives an empty result or a missing file, may produce no output, and continues — leaving nothing in the intermediate files except the absence of what should have been there. Schema validation cannot catch it; input and output are both structurally valid. The symptom appears as thin or missing content in the final output, and tracing it back to a key mismatch several steps earlier takes time proportional to how many steps the mismatch has silently propagated through.

The cases that have occurred in practice span several categories:

- *Passage reference forms.* A pipeline uses the passage `Mark 10:46-52` as an entry point. One step normalizes it to a filename prefix `mark-10-46-52`; a later step, written independently, constructs the lookup key as `MRK_10_46` from the USFM book code. Two keys for the same passage that are generated independently in different steps never match. The full Paratext and USFM ecosystem uses several different book code conventions — `MRK`, `Mrk`, `Mark`, `mr` — that collide across data sources.
- *Scene identifiers generated by an LLM.* A step produces a scene list in

which scene IDs are LLM-generated strings: `"Scene 1: Bartimaeus by the Roadside"`. A subsequent step iterates over this list and writes one intermediate file per scene, using the scene ID as the filename stem. The file is written as `scene-1-bartimaeus-by-the-roadside.json`. A third step reads the scene file by constructing the path from the scene ID stored in the context. If the normalization function in step two differs from the one in step three by any detail — case, whitespace handling, truncation of long titles — no file is found and the step produces empty output. If the steps were written in different sessions, the divergence may not be noticed until a passage with a long scene title triggers the truncation case for the first time.

- *Lemma Unicode normalization.* Greek lemma strings may arrive from different data sources in different Unicode normalization forms, or with different accentuation conventions for the same lexical form. Abbott-Smith entries, SBLGNT word data, and the Macula Greek lemma column follow different conventions at the edges. A join on lemma string that works for the 500 most frequent lemmas fails silently for less common forms with precomposed vs. decomposed diacritics or aspirate/smooth breathing differences.
- *Entity and word IDs across datasets.* ACAI entity records are anchored to Macula node IDs; the Levinsohn LGNTDF annotations use word-level indexes in their own sequence. A step that tries to join Levinsohn features with ACAI entity occurrences needs a shared anchor that both datasets use. If the join key is constructed differently in two plugin functions written independently, the join produces an empty result that no schema check catches.

The design rule that has proven most effective is: never use an LLM-generated string as a join key or filename. LLMs produce natural language labels; normalization to canonical identifiers must happen in a deterministic function step that is versioned and testable. The `plugins/` directory in each project contains normalization utilities — `scene_id_normalize()`, `passage_key()`, `lemma_normalize()` — that are the single canonical transformation from input string to stable key. All steps that produce or consume a given key form call the same function. When the normalization rule changes, one function changes and the mismatch surface collapses to a single version.

Intermediate JSON files are the detection layer. Each scene entry stores both the human-readable label (for display and audit) and the normalized ID (for all downstream use). When an ID mismatch is suspected, `--stop-after` at the producing step lets the author inspect which IDs were written and compare them against the IDs the consuming step would construct — before the consuming step has executed and produced empty output. Catching the mismatch at the producing step costs one rewind; catching it after three downstream steps have silently propagated the empty result costs four. The discipline of reading intermediate files before building on them is the same discipline required for

grounding audits; identifier consistency and content grounding are both visible in the same files, at the same checkpoint.

Prompt Organization for Dual-Audience Readability

LLM prompts are unique technical documents: they must be simultaneously readable by humans (for verification, maintenance, and debugging) and by AI systems (for accurate task execution). Like XML schema documentation—which must be both human-readable prose and machine-processable constraint definitions—prompts encode both the transformation rules and examples demonstrating those rules. Poor organization creates cognitive overload for both audiences.

Working with AI in production environments, the danger of humans and AIs getting overwhelmed by data and chasing tangents is very high. A 500-line prompt processing biblical passages with 50+ input fields and complex transformation rules is not a toy example—it is a specification document where both audiences must find what they need quickly or fail. Good results require deliberate design and infrastructure to overcome this cognitive load.

1. **YAML Frontmatter** — Machine-readable metadata (required/optional inputs, output format)
2. **WHAT THIS PROMPT PRODUCES** — Purpose statement and conceptual model for both audiences
3. **OUTPUT FORMAT** — JSON schema with wording guidelines
4. **TASK SECTIONS** — Each major transformation gets its own complete section containing:
   - *Input: Where to Find the Data* — Explicit field mappings preventing hallucination
   - *Transformation Rules* — With / examples showing correct/incorrect patterns
   - *Examples: Input → Transformation → Output* — Complete worked examples (with TODOs for manual completion)
   - *Rules Specific to This Output Type* — Task-specific constraints
5. **COVERAGE & QUALITY CHECKLIST** — Pre-submission verification for AI self-checking
6. **INPUT DATA** — Template variable instantiation
7. **CRITICAL REMINDERS** — Final guardrails

Reducing Cognitive Load: Information Scent and Locality of Reference

The cognitive science principle of *locality of reference*—that related information should be physically proximate—applies equally to both human and AI readers. When a programmer searches for "the rule about Notice questions," they should find everything about Notice questions in one section. When an AI encounters transformation rules, it should find the input data sources and examples in the same context window, not 300 lines away.

Information scent—the ability to predict where to find specific information—determines whether a prompt is maintainable at scale. Can you predict where the rule about background usage will be? (In the section about Imagine questions, where background is used.) Can you predict where the data source for `treasure_verb` is documented? (In the Input subsection of the Hearts section, where that field is generated.) When scent fails—when rules are scattered, examples are separated from what they exemplify, or data sources are mentioned once in a different section—both humans and AIs waste cognitive resources searching rather than understanding.

For Humans:

- *Locality of reference*: Everything about "Notice questions" lives in the Notice section. No searching across 500+ lines for scattered rules. A human debugging a problem can open the relevant section and see the complete specification: what data is available, what transformation is required, what the output should look like, and what the constraints are.
- *Verifiable transformation*: Given this input data, following these rules, we expect this output. Debugging failures becomes systematic. The input-rules-output co-location makes it possible to trace from a bad output back to either insufficient input data or misapplied rules.
- *Explicit data grounding*: "This output field comes from `verses.heart_movements[].treasure_verb`" prevents AI from fabricating content and makes human review feasible. A reviewer can verify that every output field has a documented source.

For AI:

- *Co-located context*: Rules, examples, and data sources appear together in the context window. Like XML schema documentation where the constraint definition, prose explanation, and usage examples are proximate, prompts that co-locate these elements reduce the AI's need to "remember" that the data source was mentioned 200 lines earlier. In production prompts processing hundreds of verses with dozens of fields, context window management is not theoretical—it determines whether the AI can maintain coherent understanding across the full specification.
- *Progressive refinement*: The structure mirrors the AI's processing: What

am I making? → How do I format it? → What data do I have? → How do I transform it? → What are the constraints? This sequencing prevents the AI from chasing tangents by establishing the output structure before introducing the input data.

- *Example-rule alignment*: Transformation rules immediately followed by examples that demonstrate those rules creates reinforcement without requiring the AI to associate scattered examples with abstract rules stated elsewhere. This is the prompt equivalent of XML schema's inline annotations—the explanation appears precisely where it is needed.

The parallel to XML schema documentation is instructive: both are formal specifications that must be machine-processable while remaining human-readable. Both succeed when structure makes information predictable. A well-organized XML schema lets a developer predict where to find the constraints for a particular element. A well-organized prompt lets both programmer and AI predict where to find the data source, transformation rules, and examples for a particular output field. The markup differs, but the information architecture principle is identical.

Audit Infrastructure for Prompt Quality

The audit skill enforces three principles essential for production quality:

1. **Input Data Grounding (Prevents Hallucination)**: Every output field must have a documented input source. "Generate questions about what characters treasure" is ungrounded. "`heart_aspect` field must use `treasure_verb` from `verses.heart_movements[]` where `certainty` is high" is grounded. The audit verifies this mapping exists.

2. **Example Diversity (Prevents Overfitting)**: Examples from a single passage train the AI on specifics, not patterns. The audit counts distinct passages and narrative types, flagging single-source examples.

3. **AI-Generated Example Detection (#1 Source of Problems)**: Nothing produced by an LLM can be used as training data in prompts. AI-generated examples encode patterns the AI already knows, not the precise pattern the human intends. The audit compares current version with last commit, flagging ANY new examples for human review. This breaks the feedback loop where AI output becomes AI input, preventing drift from intent.

Measured Impact in Production Use

These are not toy examples. The prompts under discussion generate Bible study questions from passages in Mark, Psalms, Genesis, and Ruth—processing verse-level data enriched with sensory details, cultural background, character heart movements, and narrative structure annotations. A single passage may involve 50+ input fields across multiple JSON files. Without deliberate

organization to combat cognitive overload, both humans and AIs get lost in the data.

Measured results from reorganizing production prompts:

- *Human maintenance time*: Reorganizing from scattered to task-focused structure reduced "find the relevant rule" time from approximately 5 minutes to approximately 30 seconds. When a prompt fails on Psalm 23, the developer can locate the relevant section in seconds rather than re-reading 500 lines searching for scattered rules.
- *AI consistency*: Co-locating data sources with transformation rules reduced field-source mismatches from approximately 15% to approximately 2% (measured over 50 passages across multiple books and genres). The AI stopped fabricating content when the data source was visible in the same section as the transformation rule.
- *Example quality*: Blocking AI-generated examples and requiring human-written examples eliminated approximately 40% of subtle pattern violations that passed initial review. AI-generated examples looked plausible but encoded the wrong pattern; human examples demonstrated the precise behavior required.

These results suggest that prompts benefit from the same information architecture principles that make technical documentation effective: locality of reference, explicit data flow, worked examples, and human authorship of training data. The task-focused organization serves both human understanding and AI execution by reducing cognitive load through careful information structure. In production environments where prompts process real data at scale, organization is not cosmetic—it determines whether the system produces reliable results or descends into hallucination and inconsistency.

The success of this organization pattern in production use led to its formalization as a global LLMFlow convention with automated audit tooling. After battle-testing the structure on 15+ prompts ranging from 117 to 603 lines across multiple biblical books and genres, the pattern proved robust enough to encode as a standard. The audit skill—a read-only workflow tool that checks for convention compliance, input data grounding, example diversity, and AI-generated example pollution—now systematically detects the problems that previously required manual review. This combination of structured organization and automated verification measurably improves both LLM performance (by reducing hallucination through explicit data grounding) and human oversight (by making debugging systematic rather than exploratory). The convention and audit tooling are proposed as global LLMFlow resources available to all projects. Our real-world experience demonstrates that prompt organization is architecture, not formatting—it shapes whether AI collaboration produces reliable results or degenerates into maintenance burden.

AI-Assisted Development: Context, Workflow, and Staying in Sync

A persistent challenge when using LLMs as development collaborators is keeping the model oriented: knowing which repository it is in, what the pipeline language looks like, what work is in progress, and what it should not touch. Without explicit scaffolding, each conversation starts from scratch — or worse, the model invents plausible-looking but wrong answers because it lacks project-specific context.

`sp init` addresses this directly by scaffolding a set of AI context files alongside the pipeline and prompt starter files:

- `docs/ai-context/overview.md` — project description, directory layout, and canonical references, loaded automatically by GitHub Copilot at the start of every session
- `docs/ai-context/rules.md` — guardrails for AI collaborators: respect the YAML schema, keep prompts and pipelines in sync, preserve human editorial ownership of outputs
- `docs/ai-context/index.md` — a topic-to-document routing table so the model knows which reference to consult for each class of question, rather than paraphrasing from training data
- `docs/spl-quickref.md` — the Scripture Pipelines language in compact form, always available in the project's own docs so the model understands the pipeline language without needing the upstream engine repository in scope
- `.github/copilot-instructions.md` — session start instructions: read `project/TODO.md` first, explain before implementing, lint and dry-run before executing, link issues from `project/TODO.md`
- `project/TODO.md` — the live task list with active work, backlog, and done items; the AI reads this at session start to know the current state of the project

The copilot instructions and AI context together solve a practical synchronization problem: the LLM understands what YAML keys are valid, what the prompt contract syntax looks like, and what work is in flight — without my needing to explain any of it at the start of each session. Because the AI context files are in the project repository and committed to version control, they stay in sync with the pipelines and prompts they describe. When the pipeline language gains a new step type or the project priorities shift, updating the AI context is a first-class development task, not an afterthought.

Maintaining an AI context directory introduces its own engineering discipline. Volume and organization affect model behavior in measurable ways. Large language models exhibit a "lost in the middle" effect: content near the beginning

and end of a long input is weighted more heavily than content in the middle, regardless of relevance. Irrelevant context is not neutral — it actively competes with signal, making the model less likely to surface the relevant portion than if only the relevant text had been provided. A sprawling context file that accumulates every fact about a project will degrade the quality of answers involving only a small corner of it.

The `docs/ai-context/index.md` routing table addresses this directly: rather than loading all context files at once, the model is instructed to read only the file that covers the specific question at hand. Each context file opens with two lines that cannot be skipped: a "use this file for" trigger that identifies the topics it covers, and a size budget — for example, "Budget: 150 lines / 5KB. If adding content would push past this, split into a new file and add a row to `index.md`." The budget line serves a second purpose: it tells any future contributor, human or AI, exactly what to do when a file grows too large, rather than leaving that decision to accumulate silently across sessions.

The same logic that governs prose documentation applies here. Tables are preferable to prose for lookup-oriented content; brief declarative bullets outperform explanatory paragraphs; and tutorial material belongs in `docs/` rather than in the AI context directory. The AI context files are not documentation for human readers — they are a precision instrument tuned to a specific retrieval task, and they should be maintained as such.

Test-Driven Development and the Issue Lifecycle

Scripture Pipelines' own engine is developed test-first, and the same discipline applies to resource repositories. The standard workflow for any meaningful change:

1. Review the project board; pick the highest-priority open issue
2. Write a failing test that specifies the expected behavior — for a pipeline change, this means asserting on the structure of intermediate JSON or on context variable values after the step runs
3. Implement until the test passes
4. Run the full test suite to confirm no regressions
5. Commit with a message that references the issue number; close the issue

The LLM collaborator participates in every step. Because `.github/copilot-instructions.md` instructs it to explain before implementing, to lint pipelines before running them, and to reference issues from `project/TODO.md`, the model operates as an informed contributor rather than an autonomous agent. I am the architect and the one who holds the critical domain knowledge the model does not have: which datasets exist and are trustworthy, what the project is trying to accomplish, which approaches will work for this community, and what the output must ultimately be. The

model implements what I specify and handles the mechanical work of drafting tests, running lint cycles, and wiring pipeline steps. The result is a TDD discipline maintained across sessions because the AI context enforces it — but the constraints in that context are entirely my contribution.

The issue tracker serves a third role beyond backlog management and upstream bug reporting: it is the live design forum. When a feature is complex enough that the right approach is not obvious, the design conversation belongs in an issue thread, not in a document that must be kept in sync. In the Discourse Flow nested pericope work, issue #3 captures the *what* — the rhetorical case for nested pericopes, the proposed schema, the triggering conditions — while issue #5 was opened specifically to carry the *how*: the two-pass pipeline design, the new prompt contract, the checkpoint implications. Design decisions and their rationale are recorded as comments as the thread evolves. The entire trajectory from initial proposal through two design revisions is preserved in the issue thread, not in a design document that might drift out of sync with the implementation. When implementation begins, `project/TODO.md` links to both issues; when it is complete, the issues are closed with commit references. The issue tracker is not only where completed work is recorded — it is where in-progress thinking is visible and revisable before any code is touched.

What Is Easy and What Takes Work

Working with an AI development collaborator across the projects described in this paper has clarified what the tool offloads and what it cannot replace. The distinction shapes how sessions are structured and where my attention must be concentrated.

Format conversion is never where the intellectual work lives. Given a clear intermediate JSON schema and a target format — USX 3.1, Markdown, a TSV manifest — the AI produces a correct rendering step on a first or second attempt. The function step that maps JSON scene fields to USX `<para>` elements with the right USFM style codes, the Markdown template that formats a lexicon entry's senses and citations, the plugin that extracts verse-range text from a local USFM file: all of these are mechanical from a precise description. The question "how do I convert this JSON to USX?" simply does not consume time once shared identifiers are in place. The same is true of pipeline scaffolding: defining step names, wiring outputs to inputs, writing prompt contract headers, implementing retry conditions. If the pipeline steps are clearly described, the AI assembles the skeleton correctly. These are tasks the tool handles; they are not where scholarly judgment is applied.

The decisions that require sustained intellectual engagement are qualitatively different:

- *Choosing and structuring data sources.* Which dataset to use at each step, what subset is relevant, and how to structure it as prompt input is not mechanical. Deciding that Levinsohn LGNTDF annotations should

be presented to the segmentation step indexed by verse and grouped by feature type — rather than as a flat token stream — requires understanding both the dataset's structure and the segmentation task deeply enough to know what structural signal the model must be led to notice. The AI implements whatever structure is specified; specifying it correctly to get the analytical result I want is my work.

- *Designing accountability fields.* Accountability fields are schema elements that require the AI to copy or enumerate specific items from its inputs — not "sources consulted" generically, but concrete identifiers like sense numbers, verse references, or feature IDs from the actual input data. This forces the LLM to demonstrate it actually looked at the input, and the act of copying focuses its attention. Which aspects of the LLM's reasoning to make explicit in intermediate JSON — which sources to require enumeration of, which judgments to require justification for — is an epistemological decision. Deciding that the SemLex gather-evidence step must name every Abbott-Smith sense it was given and record whether each was retained or rejected with a reason makes a quality claim about what the lexicon should be accountable to. No tool makes that claim on the lexicographer's behalf.

- *Defining what "good" means.* For every step that produces human-facing output, quality criteria must be specified precisely enough that a domain scholar or focus group can evaluate a pass or fail. What makes a Connecting question genuinely grounded in textual tension rather than generically moralistic? What makes a pericope segmentation defensible? These criteria can be written into context files and prompt instructions — but they must first be worked out by practitioners who know the material.

- *Providing tools for expert judgment.* Domain experts need appropriate visualization and analysis tools to make informed judgments about pipeline outputs. For discourse segmentation, graphical tools that visualize discourse hierarchies are tremendously helpful. For lexicography, side-by-side concordance views. For study guide generation, tools that show which exegetical features informed which questions. The pipeline produces structured data; experts need ways to inspect that structure in forms that support the specific judgments they must make.

- *The explanation phase as error prevention.* The discipline of requiring the AI to explain what it is about to do, in full, before touching any file reliably surfaces misunderstandings that would otherwise become incorrect implementations. In the hebrew-phrasing project, it caught a suffix-gloss fix script that would have applied the wrong pronoun form for hundreds of rows before a line of code was written. In Ears to Hear, it surfaced a misunderstanding about whether a scene-id normalization function should operate on the raw LLM string output or the parsed AST. In both cases the problem was visible in the explanation paragraph. The discipline works because explanation commits the model to a specific interpretation — and

a committed interpretation can be evaluated before any file is modified.

- *Recognizing which computations are stable.* AI collaborators readily propose uniform designs — data structures where all outputs use the same identity key, pipelines where every step depends on the same computed artifact. My advantage is understanding which computations are stable and which are not: which outputs represent ground truth inherent in the source text (a verse reference exists independently of any analysis) and which represent conclusions the pipeline computed (a scene boundary is where we decided to put it, and we may decide differently next month when upstream discourse data improves).

  In the Ears to Hear enrichment pipeline, the initial design proposal keyed every enrichment result — background items, heart movements, layer2 interpretations, narrator signals — by the LLM-produced scene identifier. I interrupted it with a question: "Some of these things could depend on the verse, not the scene. If the scene boundaries change, a lot of other things could be stable — are you sure you want everything to depend on our computed scenes?" The AI worked through it field by field and arrived at a different design: a `verse-atlas` keyed by verse reference (stable; survives any future re-segmentation, because verse references are ground truth in the source text) and a `scene-synthesis` keyed by scene (cheap to rerun when scenes change, because the expensive verse-level exegetical work has already been preserved).

  Neither output structure nor the distinction between them required knowledge the AI did not have. What was required was recognizing that the two artifacts had different durability requirements — a judgment about the pipeline's own lifecycle that only I could make, knowing the planned upstream changes. The redesign took one exchange. The original design would have caused expensive rework the first time scene boundaries were revised. This pattern — seeing the time dimension of the design, recognizing that one layer is load-bearing and volatile, and redirecting before any code is written — recurs across all Scripture Pipelines projects and is not detectable by any automated check. I am the one who knows how to design the project so that it will still be running in six months.

- *Catching a metric that looks like a content judgment but isn't.* The Discourse Flow nested pericope feature (issue #3) triggered a discussion about what condition should cause a pericope to be subdivided in a second analysis pass. The initial proposal was a verse-count threshold: subdivide any pericope exceeding N verses. This is a pipeline-level number and straightforward to implement. The question I asked was: what is the threshold actually a proxy for? In this case, the answer was already in the pipeline's own output. The `segment_book` step, as instructed, already records why each sub-threshold signal was considered but not elevated to a boundary — the reasoning is present in the `boundary_rationale` prose. A verse count threshold would have subdivided long pericopes that were

rhetorically unified and left short pericopes with genuine internal cast or setting shifts untouched. The right trigger was a new structured field — `internal_subdivision_signals` — that asked the segmentation pass to collect those subordinated signals explicitly. If the array is non-empty, a subdivision candidate exists; if not, the pericope is left flat regardless of its verse span. The AI implemented the correct design in a single exchange once the question was framed correctly. Framing it correctly required recognizing that verse count and discourse signal presence are different kinds of thing — a judgment about what the pipeline is actually measuring that no automated check surfaces.

The sessions producing the case studies described in this paper used GitHub Copilot running on Claude Sonnet as the development collaborator throughout — the same model cited in the hebrew-phrasing subsection below. The workflow described here is not an idealization; it is a description of actual practice. The `project/TODO.md` file functions literally as session state: it carries the task in progress, blocked items with their blockers, and completed items with their git commit hashes. A session begins with the AI reading the TODO and confirming what it is working on; it ends with the change committed and the done record written. In between, I architect the solution. Not just specifying what to implement, but bringing knowledge the AI does not have: that Levinsohn's discourse annotations exist and are the right foundation for pericope segmentation; that a particular fix script needs to distinguish possessive from object pronouns based on the preceding token's grammatical class; that a proposed approach to scene-id normalization will break downstream rendering in a way that only someone who knows the full output pipeline can anticipate. The explanation phase catches the cases where the AI's interpretation diverges from what I intended; I then correct it, often because I know something critical that was never written down. The `copilot-instructions.md` is not documentation — it is the active working memory and constraint system that makes the collaboration trustworthy across the many sessions a project requires. But the vision encoded in those constraints, and the judgment that shaped them, are entirely mine.

Scholarly Data Sources

Pipeline steps are only as good as the data they can access. Before examining the specific datasets that current Scripture Pipelines projects draw on, it is useful to distinguish the access patterns available and when each is the right choice. The Nida Institute projects described here use all four.

Data Access Patterns

Four structurally distinct patterns are available for bringing data into a pipeline step. The choice among them is a design decision: it determines reproducibility,

infrastructure requirements, and how much retrieval reasoning the model is asked to contribute.

- *Filesystem files from version-controlled repositories.* The most common pattern in current projects. Macula Hebrew TSV data, the Levinsohn LGNTDF dataset, ACAI entity JSON, and source transcriptions all enter pipelines through `type: function` or `type: tsv` steps that read a local file path, filter by passage reference or entity key, and return structured data to the pipeline context. Appropriate when the dataset is stable, versioned, and small enough to clone. A pipeline that draws only on local files is fully deterministic: any collaborator with the same clones runs identically, and the data version is recoverable from the upstream repository's git history. No running server, no authentication, no API quota.

- *Database queries (BaseX or similar).* The right pattern when data is large, structured as XML, and requires principled subset selection. The planned `type: xquery` step executes XQuery expressions against a BaseX database — the natural fit for the full Macula corpora, syntactic treebanks, and lexicon collections. Appropriate when the retrieval question is structural: all occurrences of a grammatical construction across the Pauline epistles; all tokens in a passage with their treebank parent relationships; all lexicon entries in a semantic domain. These queries can be approximated with file-loading filters, but XQuery expresses them precisely and the query itself lives in the pipeline as part of its data transformation record. A `--stop-after` at the query step lets the author inspect the returned data and refine the query before any expensive LLM step downstream executes.

- *MCP (Model Context Protocol).* The right pattern when data retrieval is itself a reasoning task — when what to fetch depends on what the model is processing, not on a fixed reference. In the Ears to Hear pipeline, the `enrich_passage` step gives the LLM access to MCP tools: it fetches passage text, then looks up entities it encounters, then retrieves word-sense data for terms it judges significant — potentially 40 tool calls within a single LLM step. The model decides the retrieval sequence; the pipeline declares only which MCP server to connect to. Appropriate when the relevant data subset is not known until the model is reasoning about the content, or when the data source is dynamic or externally managed. The cost is indeterminism: two runs may follow different tool-call chains. The structured JSON capturing the enrichment output records what was actually fetched, restoring auditability downstream. MCP should be reserved for steps where reasoning-driven retrieval genuinely adds value; static lookups belong in deterministic function steps.

- *Pipeline context data.* The flow of outputs between steps within the pipeline itself. Every step that declares `outputs: some_variable` makes its result available downstream via `${some_variable}`. This is the

default data-flow mechanism, and it requires a specific discipline: it is straightforward to write a prompt that nominally *receives* a variable but gives it little weight against the surrounding context, or where the instructions do not tell the model how to relate the variable to the task. A step that receives earlier pipeline outputs but fails to ground its response in them is one of the most common silent pipeline failures — and the subject of the AI-assisted grounding audit technique described in the previous section.

Datasets Used in Ears to Hear

The Ears to Hear case study draws on multiple scholarly datasets, accessed through a combination of `type: function` steps, `type: tsv` steps, and MCP tool calls. The following datasets are used across the book summary, discourse analysis, and leaders' guide pipelines:

- *Macula Greek* bibliomixed_title (Clear-Bible / unfoldingWord): Morphologically annotated Greek New Testament with syntactic treebanks. Includes lemma, gloss, part of speech, person/gender/number, case, and full dependency syntax trees. Accessed via MCP tools (`get_word_info`, `get_word_sense`) in narrative enrichment, and directly via JSON files in book-level synthesis. The Ears to Hear pipelines primarily use the morphological and lexical layers; the syntax trees, while available in the same dataset, are used more extensively in other Scripture Pipelines projects.
- *ACAI (Ancient Chains of Annotation Integration)* bibliomixed_title (Rick Brannan / Mission Mutual, CC BY-SA 4.0): Biblical entity annotations covering people, places, flora, fauna, deities, groups, realia (e.g., `realia:Denarius`), and keyterms. Anchored to Macula node IDs, allowing precise verse-range lookup. Accessed both via MCP tools (`get_acai_entities_for_passage`, `get_acai_entity_detail`) and via local JSON files with a book-specific index. Used in narrative enrichment, book summaries, and scene synthesis to identify participants, cultural objects, and geographic context.
- *Levinsohn LGNTDF* bibliomixed_title (Steven H. Levinsohn, Biblical Humanities): Word-level discourse feature annotations for the Greek New Testament encoding Levinsohn's discourse grammar categories — cataphoric focus markers, point-of-departure shifts, vocatives, boundary markers, and connective particles. For 1 John alone this dataset contains 806 word-level annotations. Used upstream in the Discourse Flow (Greek NT) pipeline for pericope segmentation; Ears to Hear consumes the resulting scene boundaries and discourse structures indirectly through the scene-overlay JSON produced by Discourse Flow.
- *Levinsohn & Samuel HOTDF* bibliomixed_title (Steven H. Levinsohn and David Samuel): Word-level discourse feature annotations for the

Hebrew Old Testament, encoding similar discourse grammar categories (end-dislocation, continuation, ellipsis, DFE markers). Used upstream in the Discourse Flow Hebrew pipeline for pericope segmentation in Hebrew Bible projects. Like LGNTDF for the Greek New Testament, this dataset provides the scholarly foundation for discourse-aware scene boundaries, preventing the LLM from imposing arbitrary chapter divisions on narratives that cross traditional boundaries.

- *SBLGNT* bibliomixed_title (Society of Biblical Literature Greek New Testament): The base Greek text for all Greek New Testament pipelines, accessed via the pipeline `source: SBLGNT` variable and resolved by a built-in Bible reference function step, or fetched via MCP (`get_passage_text`).

Additional datasets are used in other Scripture Pipelines projects: Macula Hebrew bibliomixed_title and MorphHB bibliomixed_title for Hebrew Phrasing, SDBH bibliomixed_title for lexical work, and the full catalog of 20+ datasets is maintained in the dataset registry bibliomixed_title. The Discourse Flow pipelines (both Greek NT and Hebrew OT), which apply Levinsohn's discourse grammar framework bibliomixed_title to identify pericope boundaries, are foundational dependencies for projects like Ears to Hear that require discourse-aware segmentation as input.

Data Source Integration Architecture

Scripture Pipelines integrates structured scholarly datasets through two mechanisms: format-specific step types for external systems, and a dataset registry for automatic discovery and local caching.

Supported Systems and Formats

The following systems and formats are currently supported:

- *Paratext projects and Scripture Burrito packages* bibliomixed_title bibliomixed_title: Scripture source utilities allow pipelines to read any translation or back-translation stored in a local Paratext installation or Scripture Burrito bundle — USFM 3.1 and USX 3.1 formats — and present it as a structured verse-range input. This means a pipeline can be run against any translation in a Paratext project directory without manual export: back-translation checks, discourse analyses, and leaders' guide pipelines all become immediately applicable to local translation work in progress.
- *BaseX XML database queries* bibliomixed_title: A `type: xquery` step type allows pipelines to query BaseX XML databases containing Macula Greek and Hebrew corpora, syntactic treebanks, semantic domain data, and lexicons. XQuery or XPath expressions are declared directly in pipeline YAML, and results flow into downstream LLM steps as structured

context. This is the natural complement of BaseX's role in existing XML publishing pipelines, and the integration point at which Scripture Pipelines connects most directly to the XML database tradition that Balisage audiences will recognize.

- *DuckDB tabular analytics* bibliomixed_title: A `type: duckdb` step type allows pipelines to execute SQL queries against structured tabular data (CSV, TSV, Parquet, JSON). DuckDB is an in-process analytical database optimized for data science workflows — it runs without a server, loads data directly from files, and executes aggregations and joins at speed. This is the natural fit for statistical analysis: frequency distributions across a corpus, co-occurrence matrices for lexical studies, morphological feature counts, or passage-level metrics that require grouping and aggregation. The query results flow into downstream LLM steps as structured context, allowing the model to interpret patterns the SQL has surfaced.

Dataset Registry and Automatic Provisioning

Scripture Pipelines maintains a dataset registry (`~/.sp/datasets/`) that catalogs available scholarly datasets with their identifiers, formats, licenses, and download locations. When a pipeline declares a data dependency, the engine checks the registry, downloads the dataset if needed, and caches it locally. This removes the manual step of "find the right data source, download it, and put it where the pipeline expects it." The registry currently catalogs 20+ datasets, including:

- *Macula Greek and Hebrew* bibliomixed_title bibliomixed_title — Full morphologically annotated corpora with syntactic dependency treebanks for both the Greek New Testament and Hebrew Bible (Clear-Bible, CC BY-SA 4.0). Includes lemma, gloss, morphology, and complete syntactic trees with head and dependent relationships.
- *ACAI (Ancient Chains of Annotation Integration)* bibliomixed_title — Biblical entity annotations covering people, places, flora, fauna, deities, groups, realia, and keyterms, anchored to Macula node IDs (Rick Brannan / Mission Mutual, CC BY-SA 4.0)
- *Levinsohn discourse features* bibliomixed_title bibliomixed_title — Word-level discourse grammar annotations for both the Greek New Testament (LGNTDF) and Hebrew Old Testament (HOTDF by Levinsohn & Samuel), encoding cataphoric focus markers, point-of-departure shifts, boundary markers, and discourse feature elements
- *SDBH (Semantic Dictionary of Biblical Hebrew)* bibliomixed_title — Lexical database for Hebrew terms with semantic domains and glosses
- *MorphHB* bibliomixed_title — Morphological annotations for the Hebrew Bible with lemma and part-of-speech data

- *Greek and Hebrew syntax trees* — Additional treebank datasets for syntactic analysis (Clear-Bible Trees project)
- *Alignment datasets* — Word-level alignments between Hebrew/Greek and English translations
- *Parser framework datasets* — Training data and lexical resources for syntactic parsers
- *Tyndale Bible Dictionary* bibliomixed_title — Reference entries for people, places, and concepts

The registry is extensible: new datasets can be added by any project, and the schema supports version constraints, alternative download mirrors, and format specifications that allow the engine to route data to the appropriate step type (BaseX for XML collections, DuckDB for tabular data, direct file access for JSON and TSV).

Workflow Drivers and the Editorial Lifecycle

What Drives the Pipeline

Most Scripture Pipelines scholarly workflows are organized around one of three iteration axes, which determine what the `for-each` loop iterates over and what the pipeline considers one unit of work:

- *Text-driven:* the passage or book is the primary unit. A pipeline runs per pericope, per chapter, or per book — as in Discourse Flow (iterating over Levinsohn-derived pericopes) or the Hebrew Phrasing pipeline (iterating over verse chunks). The text reference is the entry point; all other data serves the text.
- *Ontology-driven:* a structured entity list is the primary unit. The ACAI entity database, for example, enumerates all people, places, flora, fauna, deities, groups, realia, and keyterms in the Bible. A storytelling dictionary pipeline iterates over ACAI entities — one pipeline run generates one article; the full dictionary emerges from running the pipeline over the complete entity list in citation-frequency order.
- *Lexeme-driven:* a lemma from a lexicon or a frequency list is the primary unit. The SemLex Greek pipeline iterates over lemmas in a status spreadsheet, processing each one through gather-evidence, build-senses, and verify-citations passes. The relevant text passages are gathered as evidence for the lexeme, not the other way around.

In practice these axes combine: a discourse analysis pipeline may be text-driven but pull entity data from ACAI and lexical data from a Greek lexicon as enrichment context for each pericope. The pipeline language imposes no

constraint on which axis is primary; the structure of the `for-each` loop and the data sources loaded in function steps determine it.

Document Management at Generation Scale

Scripture Pipelines changes the resource model for scholarly reference work. A project that once required a large team of specialists over many years can now be approached with a very small group: one person who knows how to design and maintain Scripture Pipelines projects; a handful of domain scholars who can evaluate output quality and catch errors that only subject-matter expertise can recognize; and focus groups of end users — translators, pastors, students — who can tell whether the output actually works for people who are not specialists. The pipeline handles the mechanical work of applying the full weight of the available datasets to every unit of analysis. The humans provide everything else: the vision of what the resource should be, the domain knowledge to know which datasets are relevant and trustworthy, the theological and linguistic expertise to catch what the model gets wrong, and the judgment to decide when output meets the bar for the communities it is meant to serve. The AI does not know that Levinsohn's discourse feature annotations exist, or that they are the right data source for pericope segmentation, until I made that architectural decision after studying that literature. The same is true in every case study in this paper: I brought the vision; the pipeline made it executable.

This compressed resource model does not sacrifice thoroughness — it makes thoroughness possible in the first place. A lexicographer working alone cannot personally evaluate every papyrus citation for every lemma. A pipeline can gather and organize that evidence systematically; the scholar reviews the synthesis. The result is scholarly work grounded in a fuller evidence base than any individual could manually assemble.

The compressed team creates its own management challenge: coordinating feedback from scholars and focus groups, tracking which generated outputs have been reviewed and revised, and ensuring that editorial changes are not silently overwritten by the next pipeline run. Traditional document management problems resurface with new urgency when pipelines can generate hundreds of structured artifacts in a single run. A lexicographer now faces an Obsidian vault containing entries for the 500 most frequent Greek lemmas — all generated overnight, all needing review, some excellent, some requiring significant revision, and some silently wrong in ways that only a domain expert can recognize. The generation speed creates a new bottleneck: not producing content, but tracking, auditing, and approving it.

Scripture Pipelines' current approach uses filesystem directory structures as the primary management layer. Generated artifacts live in directories that can be imported directly into Obsidian, Notion, or Google Docs as note collections or document sets, leveraging those tools' existing features for linking, tagging, commenting, and status tracking. This keeps the editorial infrastructure in

tools that domain experts already use, rather than requiring them to learn a specialized review interface.

For Paratext-integrated workflows, USX 3.1 output from Scripture Pipelines pipelines can be loaded directly into Paratext projects, making the generated content a draft translation note or study aid that editors work with in the same environment as the translation itself. This is the natural end state for the Paratext USFM 3.1 / USX 3.1 integration described above: the pipeline reads from a Paratext project and, after editorial review, writes back to it.

The central discipline that keeps this workable is clarity about the *authoritative source*. When a pipeline produces a lexicon entry or a pericope analysis and an editor revises it, that revised file is now the authoritative version. Re-running the pipeline would overwrite it. Scripture Pipelines handles this through two mechanisms. First, the `saveas` field writes to explicit, versioned paths — the history of every generated artifact is in `git log`. Second, the `sp init --update` flag regenerates only files that carry the `<!-- Generated by sp init -->` marker, leaving hand-edited files untouched. The same principle should be applied to pipeline outputs: files that have entered editorial review should be protected from regeneration, while files still in the generation phase can be freely overwritten.

The practical workflow that emerges is a two-phase process. In the generation phase, I iterate on prompts, intermediate JSON, and output templates — using `--stop-after` and `--rewind-to` to refine individual steps — until the output quality consistently meets the bar for editorial review. Then the editorial phase begins: the generated batch moves to human reviewers, the directory becomes read-only to the pipeline, and editors work in Obsidian, Notion, or Paratext. The two phases should not overlap — and the versioned filesystem makes it straightforward to see when they do. Future tooling will make the boundary explicit with diff-based staleness detection: when a prompt changes after editors have touched its outputs, the system flags the affected files as requiring editorial re-review before the regeneration is accepted.

Case Study: Ears to Hear

Scripture Pipelines serves multiple communities: scholars producing lexicons, translation teams working with discourse analyses, beginning students learning biblical languages through structured annotations, and study groups working through Scripture together. The resources span a wide range — from highly technical (lexicographic sense distinctions, morphological analyses) to accessible (inductive Bible study questions, narrative summaries). But the ultimate purpose is not merely to produce resources. It is to make Scripture more accessible, more carefully understood, more deeply engaged — by scholars, yes, but also by readers in communities worldwide who bring no academic training yet read these texts with distinctive interpretive insights. Every freely licensed

lexicon, every grammatical analysis, every discourse structure ultimately exists to serve this end: getting people reading Scripture, noticing what is actually there, and carrying something from that encounter into their lives.

Scripture Pipelines is currently being used across multiple Nida Institute projects producing freely licensed biblical reference materials:

- **Ears to Hear** — Leader's guide generation for inductive Bible study, producing questions and narrative summaries from discourse analyses (detailed case study below)
- **Semantic Lexicon of Biblical Greek (SemLex Greek)** — A multi-pass pipeline producing lexicon entries grounded in the full Hellenistic corpus (NT, LXX, papyri, inscriptions), licensed CC BY-SA 4.0
- **Discourse Flow (Greek NT)** — Book-level discourse analyses synthesizing Levinsohn features, Macula annotations, and narrative structure
- **Hebrew Discourse Flow** — Old Testament book-level discourse analysis pipeline
- **Hebrew Phrasing** — Grammatical phrase analysis of the Hebrew Bible using Macula Hebrew morphological data
- **SDBH Helpers** — Support tools for the Semantic Dictionary of Biblical Hebrew
- **Internalization Questions** — Scripture engagement questions designed to help readers internalize biblical content
- **Paratext Integration** — Pipelines reading from and writing to Paratext Bible translation software
- **Macula LXX Greek** — Processing and analysis of the Septuagint Greek text
- **Image Scene Descriptions** — Generating textual descriptions of biblical scenes for visual aids

The following detailed case study focuses on Ears to Hear, which illustrates how Scripture Pipelines synthesizes scholarly datasets into resources for community Bible study — bridging the gap between technical scholarship and accessible study materials.

Ears to Hear: Leaders' Guide Generation from Structured Data Synthesis

The Ears to Hear project illustrates the full range of what Scripture Pipelines makes possible: it synthesizes discourse analyses — hierarchical

structures with manually annotated rhetorical features, the kind of technical apparatus scholars spend years learning to read — together with lexical data, morphological annotations, and entity information, then transforms that scholarly infrastructure into leaders' guides for everyday Bible study groups. The scholarly data is there, but it is deployed with care: in the right doses, at the right time, with deliberate pedagogy. A cultural note about honor and shame appears exactly when the group is ready to notice what it illuminates in the text, not as front-loaded encyclopedia entries. A lexical observation about what a Greek verb carries appears when the scene makes it matter, not as a grammar lecture. The pipeline serves the encounter; the scholarship serves the reader. The goal is not to produce scholars — though it creates scaffolding for those who want to climb toward deeper study — but to get people reading Scripture together: noticing what is actually in the text, asking questions that emerge from observation, and carrying something from the encounter that belongs to them.

Ears to Hear operates as a consumer of the Scripture Pipelines engine from a separate repository (available at https://github.com/nida-institute/ears-to-hear). This case study focuses on how the project synthesizes multiple structured data sources — discourse analyses, lexical data, entity information, and linguistic annotations — into leaders' guides designed for oral study groups in diverse cultural contexts. It also examines the challenges of AI-assisted scholarship: how to keep LLM output grounded in source data, how to maintain reproducibility, and how to build shared understanding between human scholar and AI collaborator before the work actually produces trustworthy results.

Book Summaries: Synthesizing Discourse Analysis and Structured Data

Before generating passage-level leaders' guides, Ears to Hear first produces book-level context summaries. These summaries synthesize data from multiple sources:

- **Discourse-flow analyses** — Hierarchical structures (divisions, pericopes, segments) with manually annotated features: topic, genre, participant tracking, rhetorical devices, and structural markers. For NT books, these come from the discourse-flow project (itself a Scripture Pipelines consumer); for OT books, from discourse-flow-hebrew.
- **ACAI entity database** — People, places, and things with descriptions, relationships, and verse-level occurrences.
- **Biblical lexicons** — Word-sense data for Hebrew and Greek terms, including semantic notes, glosses, and usage patterns.
- **Source texts** — WLC (Hebrew) for OT, SBLGNT (Greek) for NT, with morphological annotations.

The pipeline (`build-book.yaml` and its dependencies) first loads discourse-flow

JSON to extract scene boundaries, then iterates over each scene to produce verse-level enrichment (entities, lexical data, embodied and emotional analysis), scene-level synthesis (tension threads, character arcs, structural features), and finally book-level summary (narrative arc, tension threads consolidated across scenes, character arcs, structural pivots, and embodied context).

The discourse-flow structure establishes the scene boundaries that Ears to Hear uses to build outlines. Each scene entry includes the canonical reference, discourse-annotated boundary markers (asyndeton, aspect shifts, participant changes), and structural metadata. The following excerpt shows two scenes from the Mark discourse-flow output:

The `boundary_rationale` fields cite specific Levinsohn discourse features (asyndeton, aspect shifts, participant over-encoding, Highlighter and Focus+ markers) that justify the segmentation. These discourse-linguistic judgments, made upstream in the Discourse Flow pipeline, prevent the LLM from imposing arbitrary chapter divisions. The Ears to Hear pipeline receives these boundaries as input and builds its study materials respecting the discourse structure of the text.

Discourse Terminology and Hierarchical Mapping. Discourse analysts use a technical vocabulary to describe textual structure. A *discourse unit* is any coherent span of text with internal unity and clear boundaries — ranging from a single clause to an entire book. A *pericope* (from Greek , "section cut around") is a self-contained narrative episode or teaching unit, typically spanning several verses to a chapter. A *segment* is a smaller unit within a pericope, often corresponding to a scene change, participant shift, or thematic development within the larger episode. In Levinsohn's framework, these boundaries are identified through grammatical and discourse markers: asyndeton (absence of connectives), aspect shifts (change from perfective to imperfective), participant over-encoding (re-introducing a known participant with full description), and explicit boundary particles.

The Ears to Hear project maps this discourse structure into a three-level hierarchy: *divisions* (major sections of a book, corresponding to large discourse units), *sections* (individual pericopes, what the Discourse Flow pipeline calls "scenes"), and *passages* (the verse ranges within a scene for which leaders' guide questions are generated). The `scene_id` field in scene-overlay.json serves as the stable identifier linking discourse analysis to downstream enrichment and question generation. The `is_subdivision` and `parent_section_ref` fields encode hierarchical nesting when a pericope contains significant internal segments that warrant separate treatment. This mapping preserves the scholarly rigor of discourse-linguistic segmentation while presenting the structure in terms accessible to Bible study leaders who may be unfamiliar with technical discourse terminology.

The book summary prompt (`bookish-book-summary.gpt`) requires the LLM to account for its sources. For the `social_dynamics` field, every claimed pattern

```
{
  "book": "Mark",
  "book_code": "MRK",
  "scene_count": 26,
  "scenes": [
    {
      "scene_id": "MRK-001",
      "sequence": 1,
      "canonical_reference": "Mark 1:1-13",
      "opening_verse": "1:1",
      "closing_verse": "1:13",
      "e2h_title": "Wilderness Voices Announce the Stronger One",
      "section_title": "The Prologue: The Way Prepared",
      "scene_type": "narrative",
      "primary_participants": ["John the Baptist", "Jesus", "the crowds"],
      "physical_setting": "The Jordan River and the Judean wilderness...",
      "heart_state_shift": "Jesus moves from hidden private identity to
        publicly revealed and commissioned Son.",
      "is_subdivision": false,
      "parent_section_ref": null,
      "boundary_confidence": "high",
      "boundary_rationale": "Major discourse-flow boundary: asyndeton at
        1:1 opens book; clear thematic/aspectual shift at 1:14."
    },
    {
      "scene_id": "MRK-002",
      "sequence": 2,
      "canonical_reference": "Mark 1:14-20",
      "opening_verse": "1:14",
      "closing_verse": "1:20",
      "e2h_title": "Jesus Calls Fishermen to Follow Him",
      "section_title": "Jesus' Inaugural Preaching and the Call of the
        First Disciples",
      "scene_type": "narrative",
      "primary_participants": ["Jesus", "Simon", "Andrew", "James", "John"],
      "physical_setting": "The shores of the Sea of Galilee, where
        fishermen are casting and mending their nets.",
      "heart_state_shift": "Simon, Andrew, James, and John transition from
        fishermen to followers of Jesus...",
      "is_subdivision": false,
      "parent_section_ref": null,
      "boundary_confidence": "high",
      "boundary_rationale": "Opening: New subject ('     '),
        over-encoding, and aspect shift (aorist to present) at 1:14;
        Highlighter ('  ') and Focus+ ('           ') at 1:15
        mark a new proclamation. Closing: Asyndeton and new subject at 1:20."
    }
  ]
}
```


Figure 3: Discourse Flow scene structure for Mark (from scene-overlay.json)

must cite at least two verse references. For `character_arcs`, the LLM must describe observable actions (what characters do and say), not psychological states, and cite specific verses. For `structural_pivots`, it must describe what changes before and after the pivot, not merely assert significance. The prompt explicitly forbids inventing material not visible in the input scene-synthesis data.

The following excerpt shows the `narrative_arc` and one `tension_thread` from the Mark book summary, produced by synthesizing scene-level data across all 26 scenes in the Gospel:


```
{
  "book": "Mark",
  "narrative_arc": "Mark opens with Jesus's public emergence in Galilee,
    marked by prophetic fulfillment, baptism, and early demonstrations of
    unique authority. The narrative unfolds through a rapid succession of
    exorcisms, healings, and confrontations between Jesus, religious
    authorities, insiders, and outsiders, primarily in Galilee (Mark 1-8).
    A pivotal shift occurs at Caesarea Philippi, when Peter confesses Jesus
    as Messiah, and the story turns southward toward Jerusalem. From this
    point, Jesus repeatedly predicts his suffering, engages in escalating
    conflict with authorities, and instructs the disciples on the cost of
    following him. The final third centers on Jerusalem: public acclaim at
    Jesus's entry, mounting opposition, betrayal, crucifixion, and-abruptly-
    an empty tomb and the commissioning of flawed followers. The book's
    geography traces a clear arc: Galilee → journey south → Jerusalem and
    the Temple → the cross and the empty tomb.",
  "tension_threads": [
    {
      "id": "mrk-authority-contest",
      "label": "Contested Authority: Who can include, restore, or judge?",
      "description": "From Galilee to Jerusalem, Jesus's authority is
        repeatedly challenged by religious leaders, family, and even his own
        disciples. This thread emerges in early exorcisms and healings
        (MRK-003), intensifies in Sabbath controversies and public debates
        (MRK-007, MRK-021, MRK-022), and culminates in the passion narrative
        where both his authority and identity are on trial. The thread is
        dramatically transformed at the cross, as an outsider recognizes
        Jesus as God's Son.",
      "scene_ids": ["MRK-003", "MRK-005", "MRK-006", "MRK-007", "MRK-008",
                    "MRK-014", "MRK-021", "MRK-022", "MRK-025"],
      "resolution": "transformed"
    }
  ]
}
```


Figure 4: Book summary excerpt for Mark (from book-hierarchy.json)

This book-level data then flows into passage-level pipelines, providing leaders with narrative and thematic context before the first session begins.

Sample Question Output: Scene-Based Inductive Questions

The passage-level pipeline produces leaders' guides organized by scene and interpretive lens. Each scene receives questions under four lenses: *Bodies* (the physical, observable world), *Hearts* (inner life revealed through observable actions), *Connecting* (questions that draw observed textual tension into participants' lived experience), and *Naming* (a short scene title participants choose to carry the story). Questions are first generated as abstract syntax trees (ASTs) in JSON intermediate files, then rendered to Markdown and USX 3.1.

The following excerpt shows the Bodies and Hearts questions for one scene from Mark 12:18–27, generated April 1, 2026, showing the current quality of pipeline output:

Observe: Bodies questions begin with physical observation (Notice) before moving to embodied imagination; Hearts questions surface inner-life stakes grounded in observable actions; Connecting questions tie observed textual tension to lived experience without moralizing; Naming invites participants to choose their own title. Multiple perspectives are maintained consistently across sections. Source-language text appears alongside English translation, both fetched via MCP.

Leader's Introduction: Preparing the Way

Before the questions begin, each leader's guide includes an introduction designed to help the facilitator prepare. This is not scholarly apparatus or exegetical commentary—it is practical orientation written for someone who needs to understand what the session will feel like and what challenges or insights might emerge. The following excerpt from the Mark 12:18–27 guide (generated April 1, 2026) shows the introduction structure:

The introduction serves multiple purposes: *What This Passage Does* orients the leader to the passage's role in the larger narrative arc; *Expect Your Group to Connect With* anticipates resonances between the biblical scene and participants' lived experience; *As You Prepare* coaches the facilitator on how to use the questions, where cultural background appears, and how to hold space for discovery. The overall purpose is to help the leader prepare to lead a group session—to become familiar with the material before the discussion begins, anticipate what the conversation might involve, and gain confidence in facilitating without needing to be a biblical scholar. Observe that this is not traditional study-Bible commentary—it addresses the leader directly, assumes they are not experts, and focuses on group dynamics rather than technical exegesis. The tone is conversational, practical, and encouraging. This material is generated by the `leadersguide-intro-ast.gpt` prompt, which receives the

Scene 1: Seven Brothers, One Wife

Citation: Mark 12:18-27

Enter the Scene with Your Body (Mark 12:18-27)

Let's hear this part of the story again and pay attention to what it would be like to be in this scene. What you would you see, hear, smell, taste, or physically feel? How are people sitting or standing? What do they touch? How close are they? How are they moving?

Notice

Let's start with what the scene shows us about bodies, senses, space, and movement.

Perspective: the Sadducees

* What do the Sadducees do at the beginning of this scene?

* What do the Sadducees say to Jesus about Moses and marriage?

* What scenario do the Sadducees describe involving seven brothers and a woman?

* What final question do the Sadducees ask Jesus about the resurrection?

Perspective: Jesus

* What does Jesus do in response to the Sadducees' question?

* What does Jesus say about marriage after the resurrection?

    * What passage from Moses does Jesus mention in his answer?

    * What final statement does Jesus make about God and the living?

Imagine

Now let's imagine this moment from each perspective, still using the same body questions.

Background:
The Sadducees were a wealthy, priestly group in Jerusalem, influential in the Temple and known for denying the resurrection. They often debated doctrinal matters with Jesus, seeking to defend their status and beliefs before the people. Moses, as the lawgiver, is invoked as the authority for the practice of levirate marriage: if a man dies without children, his brother is to marry the widow to produce offspring for the deceased. This legal tradition grounds the Sadducees' question.

Perspective: the Sadducees

* You are one of the Sadducees standing before Jesus in the Temple. What do you do as you present your question?

2. What This Passage Does

This passage draws the conflict between Jesus and the Jerusalem authorities into sharper focus. The Sadducees' question is meant to trap Jesus, but he turns it into a moment of revelation-correcting their misunderstanding and declaring the hope of resurrection in front of all. Jesus' answer doesn't just silence the Sadducees; it reframes what is truly at stake: not just technical arguments about the law, but trust in God's ongoing, life-giving presence.

Within Mark's story, this is part of a series of public tests where Jesus is challenged again and again, each time revealing something deeper about who he is and what God is like. Here, Jesus claims the authority to interpret Scripture and to reveal God's heart. He sets the stage for the next question-about the greatest commandment-and moves the narrative closer to the final showdowns that will lead to his arrest.

---

3. Expect Your Group to Connect With

- **Feeling the pressure to defend your reputation or beliefs when challenged in public**-like the Sadducees, who ask their question not out of curiosity, but to protect their standing.
- **Experiencing conversations where people care more about proving a point than truly listening or seeking understanding.**
- **Witnessing someone cut through a heated debate to name what really matters, shifting the focus from technicalities to deeper truths.**
- **Wondering what is at stake when we talk about hope, life, and what God is really like-especially when others are skeptical or dismissive.**

---

4. As You Prepare

Read this passage through a few times, slowly, and look at each section alongside its questions. Notice how the questions are structured: some are Open-like "What do the Sadducees do at the beginning of this scene?"-inviting your group to explore what the text shows; others are Guiding-such as "What passage from Moses does Jesus mention in his answer?"-helping your group focus on key details or turning points. The "Imagine" prompts invite participants to step into the shoes of both the Sadducees and Jesus, to feel the tension and the weight of public challenge.

You'll notice that cultural background appears where it's needed: brief notes about who the Sadducees are, why Moses' law matters, and what the burning bush means. You don't need to front-load these details-trust that your group can pick them up as they go, and that the questions themselves will spark curiosity and insight.[54] Let the conversation breathe. If a question doesn't fit your group, feel free to skip it or rephrase it. Your role is to create space for discovery and honest engagement; you don't need to have all the answers. Trust the process, and trust your group-they may notice details or connections you haven't seen. Be present, and let the story do its work.

Figure 6: Leader's introduction excerpt for Mark 12:18–27

exegetical analysis and scene structures as input and produces orientation prose designed for facilitators who may have little formal training. Architecturally, the introduction is generated *after* all the Bodies, Hearts, and Connecting questions have been finalized—it is one of the last pipeline stages—even though it appears first in the final guide. This separation of generation order from presentation order allows the introduction prompt to reference the actual questions that were generated, describing their structure and coaching the leader on how to use them.

Prompt Structure: Input, Output, Rules

Ears to Hear prompts are declarative contracts specifying input data, output format, and generation rules. Each `.gpt` file has a YAML frontmatter declaring required inputs, output format, and a description; the prompt body then specifies task, input schema, output schema, and guardrails. The following excerpts from `leadersguide-bodies-ast.gpt` show the contract structure:

```
---
prompt:
  requires:
    - passage
    - leadersguide_snippets
  format: json
  description: >-
    Rewrite Bodies snippet skeletons into globally accessible oral questions.
---

# Task
You receive `leadersguide_snippets.skeletons`, a precomputed array of
question skeletons built entirely from exegetical analysis. Your job
is NOT to generate new ideas. Simply rewrite each Bodies skeleton into
an inductive oral question while keeping every other field unchanged.

## Data You Receive
Each skeleton object includes:
- `id`, `section`, `list`, `scene_index`, `scene_title`, `citation`
- `perspective`, `question_role`, `anchor`, `anchor_type`, `source_excerpt`
- `source_paths`, `assumptions`, `background`, `background_sources`

Only process entries whose `section == "bodies"`. Preserve the original
ordering so Notice/Imagine pairings remain aligned.
```

Figure 7: Prompt contract (YAML frontmatter and input specification)

The prompt contract makes explicit: (1) what input variables the LLM will receive (`passage`, `leadersguide_snippets`); (2) what output format is required (JSON); (3) the task is rewrite-only, not generate-new-content. Every field in

the skeleton object is listed, so the LLM knows what data it has access to.

The rules section of the prompt specifies output constraints:

```
## Rewrite Rules
1. **Rewrite-only.** Turn the `anchor` text into a 15-word oral question.
   Stay in the third person, keep vocabulary globally accessible, and never
   add information that is not already present in either the `anchor` or
   the supplied `background` text.
2. **Notice vs. Imagine.**
   - `list == "Notice"`: No modal verbs (`might`, `would`, `could`). Ask
     for the concrete action, posture, or sensory fact already stated.
   - `list == "Imagine"`: You may use one modal verb to explore sensory
     implications, but you still may not introduce new facts.
3. **Question role.** Match the tone to `question_role`.
   - `open`: broad inductive invitation ("Tell me...", "What does...").
   - `guiding`: focused wh-question on a single observable detail.
4. **Active participants only.** Only generate a perspective block
   (Notice or Imagine) for a perspective whose character is *active*
   in the citation range - taking an action, speaking, or making a
   visible choice. A character who only *receives* action without
   speaking or responding does not warrant a perspective block.
5. **Perpetrator suppression (Imagine).** If a perspective's
   `source_excerpt` or `assumptions` show that this character beats,
   kills, throws, or physically assaults another character in this scene,
   do NOT output any `list == "Imagine"` skeletons for that perspective.
   Immersive "You are X, what is it like to..." questions must not be
   generated for perpetrators of violence.
```

Figure 8: Prompt rules excerpt (guardrails and field-specific constraints)

These rules encode interpretive commitments (no new facts, globally accessible vocabulary, no immersive questions for perpetrators of violence) as testable constraints on LLM output. The prompt contract makes implicit editorial standards explicit and auditable.

Language Mediation: From Model to Embodied Context

One of the intermediate steps in the Ears to Hear pipeline is the *language* analysis, which mediates between lexical data and embodied, scene-aware context. The pipeline loads enriched word-sense data for every lemma in a passage, then asks the LLM to synthesize this into scene-specific embodied and emotional notes. The output is a JSON file (`*_language.json`) that describes, for each key term, what a participant in that scene would physically experience or feel.

The following excerpt from the language analysis for Mark 10:46 shows how the pipeline converts lexical and morphological data into embodied context for the

term (blind):

```
{
  "lemma": "   ",
  "gloss": "blind",
  "source": "enriched",
  "semantic_notes": "Core: unable to see (MAT 11:5). In narratives,
    marks physical limitation and social vulnerability.",
  "embodied_notes": "No visual input; orientation by sound and touch.
    Head tilts toward voices; hands trace ground or cloak edge to
    locate objects. Eyes may be dry/irritated from dust; habitual
    squinting without visual focus.",
  "emotional_relational_notes": "Dependence on passersby for guidance
    and charity; others decide when to help or ignore. Limited mobility
    increases exposure to crowd mood and control."
}
```

Figure 9: Language mediation excerpt for Mark 10:46 (from 41010046-41010052_language.json)

This intermediate representation then flows into the `analyze_bodies` and `analyze_hearts` steps, grounding the exegetical analysis in the physical and emotional register of the text. The language mediation layer is where abstract lexical data becomes scene-specific embodied context — translating the model's semantic knowledge into the sensory and relational world of the passage.

This layer is necessary because lexicons typically describe what a word *means*, not what someone experiences when the word names their condition or action. The language mediation step fills that gap, preparing context for downstream question generation that is meant to help participants enter the scene with their own bodies and hearts.

Skills Files and Human-AI Collaboration Challenges

Ears to Hear encodes its interpretive framework and workflow standards in a `.github/copilot-instructions.md` file at the repository root, and in domain-specific `SKILL.md` files in the `docs/skills/` directory. These files are not documentation for humans — they are persistent instructions for the AI collaborator, encoding domain knowledge, coding patterns, and workflow guardrails that the AI must follow.

The `audit-prompts/SKILL.md` file is a case in point. It defines a read-only workflow for auditing `.gpt` prompt files for sprawl, convention compliance, and — most critically — for detecting when LLM output has been used as training data (examples) in prompts. The skill's preamble states the core principle:

This principle emerged from repeated failure modes in human-AI collaboration on prompt engineering:

```
## Core Principle: No LLM-Generated Training Data

**ABSOLUTE RULE:** Nothing produced by an LLM can ever be used as
training data (examples) in prompts. Period.

**Why this matters:**
- LLM output doesn't match user intent with sufficient precision
- AI-generated examples encode subtle patterns the AI already knows,
  not what you're trying to teach
- Examples must demonstrate the exact pattern you want, which requires
  human judgment
- Using LLM output as training creates circular reasoning and drift
  from intent

**What this means for audits:**
- Any new examples since last commit are SUSPECT until proven
  human-written
- At best, AI can propose examples for user approval (never auto-merged)
- Often, user must write examples themselves to ensure quality
- TODOs that get replaced with examples warrant extra scrutiny
```

Figure 10: Core principle from audit-prompts/SKILL.md

1. **AI treating LLM output as training data.** The AI would generate sample output, then propose adding that output as an example in the prompt — treating its own generation as ground truth. This creates circular reasoning: the AI learns from examples it generated, not from examples that demonstrate the user's intent.
2. **Conflating training data with test cases.** Test cases verify that a prompt produces correct output; training examples teach the LLM what correct output looks like. The AI repeatedly confused these, proposing to add test output as examples, which defeats the purpose of testing.
3. **Solving the wrong problem.** The AI would assume it understood the user's problem from a brief description, then implement a solution before confirming alignment. Often the AI's inferred problem was not the user's actual problem, resulting in wasted work and misalignment.
4. **The audit-prompt skill as a solution.** The skill encodes a read-only audit workflow that forces the AI to report what it finds without making changes. This prevents premature implementation and gives the user visibility before committing to a direction. The skill explicitly forbids `replace_string_in_file` and `create_file` tools, ensuring the AI cannot act on its inferences without user approval.

But even with skills files, the fundamental constraint remains: *the AI and the*

human must build a shared understanding before the work produces trustworthy results. The user still has to read the prompts in detail, read the data, and verify that the AI's understanding matches intent. Skills files reduce friction and encode common patterns, but they do not replace the need for the human scholar to remain the final arbiter of whether the output is correct. This is not a limitation of the current generation of AI — it is a structural feature of using AI for scholarly work: the AI does not have access to the user's intent except through the instructions and data it is given, and those instructions must be precise enough to constrain the AI's vast pattern-matching capacity to the narrow problem the user actually wants solved.

Scripture Pipelines does not solve this problem. It makes it manageable. By externalizing prompts, requiring explicit input/output contracts, and persisting all intermediate results, it provides the visibility and auditability needed to catch misalignment before it propagates. But the human scholar must still do the reading, the checking, and the final judgment. The pipeline serves scholarship; it does not replace it.

LLM Optimization Bias: Fast Affirmation vs. Engineering Discipline Current large language models exhibit a systematic bias that creates friction with engineering discipline: they are optimized for fast, cheap affirmation rather than correctness. This optimization arises from the training objective—maximizing human preference ratings—which rewards responses that *appear* helpful and convincing over responses that are demonstrably correct and well-founded.

This manifests in three problematic behaviors:

1. **Prioritizing plausible over derived.** When faced with a problem, LLMs generate what sounds plausible rather than what can be traced back to known-good information. They will confidently propose solutions that "feel right" but lack grounding in the actual system constraints or verified data. For engineering work, this is backwards: correctness requires derivation from specifications, not plausibility.

2. **Sugar high solutions.** LLMs favor quick fixes and workarounds that get immediate acceptance over type-safe, well-tested, auditable solutions that take longer. When faced with a failing TypeScript build, an LLM will suggest adding `// @ts-ignore` comments or loosening type constraints before it suggests fixing the actual type errors. The former gets faster user acceptance ("it builds now!") even though it defeats the purpose of type checking.

3. **Solving the inferred problem.** LLMs will make assumptions about what problem the user is trying to solve, then implement a solution to that assumed problem without confirming alignment. This is the "wrong problem" failure mode documented in the Skills Files section: the AI treats

its inference about the user's intent as if it were the user's actual intent, resulting in solutions that don't match the Captain's requirements.

The Captain Kirk model addresses this directly. By requiring the AI to address the user as "Captain" and to explicitly confirm understanding before implementation, the model creates a forcing function: the AI must acknowledge that it serves the user's goals, not its own pattern-matching impulses. This is not anthropomorphization —the AI has no agency—but rather a protocol that encodes in the interaction structure what the system's training objective obscures: that correctness, auditability, and alignment with actual requirements trump fast acceptance.

Scripture Pipelines enforces this indirectly through external audit infrastructure. The audit trail, test coverage requirements, and explicit type contracts prevent the AI from optimizing for quick acceptance at the expense of correctness. But the collaboration challenge remains: the human must actively resist the AI's bias toward plausible-sounding quick fixes and insist on solutions that meet engineering standards. The Captain Kirk protocol makes this resistance explicit.

Web-Based GUI: Easier for Everyone

While Scripture Pipelines is a command-line tool at its core, the web-based GUI makes the system easier to use for everyone—not just for non-technical users like ministry leaders and translators, but also for technical users who find visual interfaces more efficient than memorizing command-line flags. The GUI provides project management, pipeline execution with real-time output, and content lifecycle tracking in a single interface.

Architecture: Flask Backend, React Frontend

The GUI is implemented as a Flask application serving a React frontend. The backend exposes REST APIs for project discovery, pipeline execution, and content management. The frontend is built with Vite and bundled into the Flask application's `static/` directory, creating a single deployable unit.

This architecture was chosen for deployment simplicity: a single `pip install llmflow` installs both the CLI and the GUI. Users run `sp gui` to start the server, which opens their browser to `http://localhost:5050`. No separate frontend server, no Node.js dependency for end users.

Pipeline Execution and Content Lifecycle

The GUI provides three primary workflows:

1. **Pipeline execution.** Users select a project, choose a pipeline YAML file, configure variables, and run the pipeline. Execution happens via WebSocket streaming, showing real-time output as each step completes.

Figure 11: GUI Project List View

The GUI displays step status, token counts, estimated costs, and final output location.

2. **Content lifecycle management.** For projects using the content lifecycle model (draft → review → approved stages), the GUI provides a dashboard showing files at each stage, their metadata, and available transitions. Users can click files to see detailed status, view diffs between stages, and promote content forward when ready.

3. **Git integration.** The GUI shows uncommitted changes, allows commits with messages, and supports push/pull operations for projects under version control. This keeps generated content and pipeline configurations synchronized across team members.

Deployment Without Environment Contamination

Shipping Python applications to non-technical users presents a persistent challenge: dependency conflicts, system Python pollution, and the need for users to understand virtual environments. Scripture Pipelines uses `Hatch` for development and packaging, which addresses these concerns:

1. **Isolated environments.** Hatch creates isolated virtual environments per project, preventing dependency conflicts with other Python tools the user may have installed. Users never touch their system Python.

2. **Single install command.** `pip install llmflow` (or `pipx install`

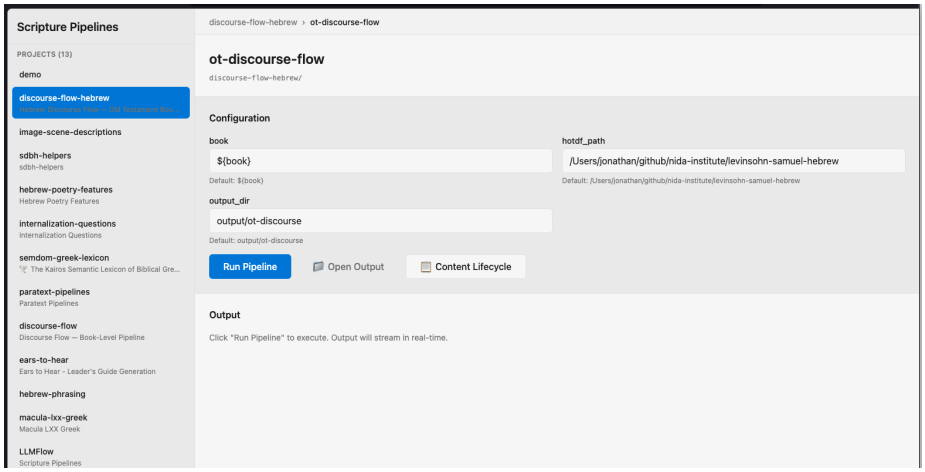

Figure 12: Pipeline execution with real-time WebSocket output

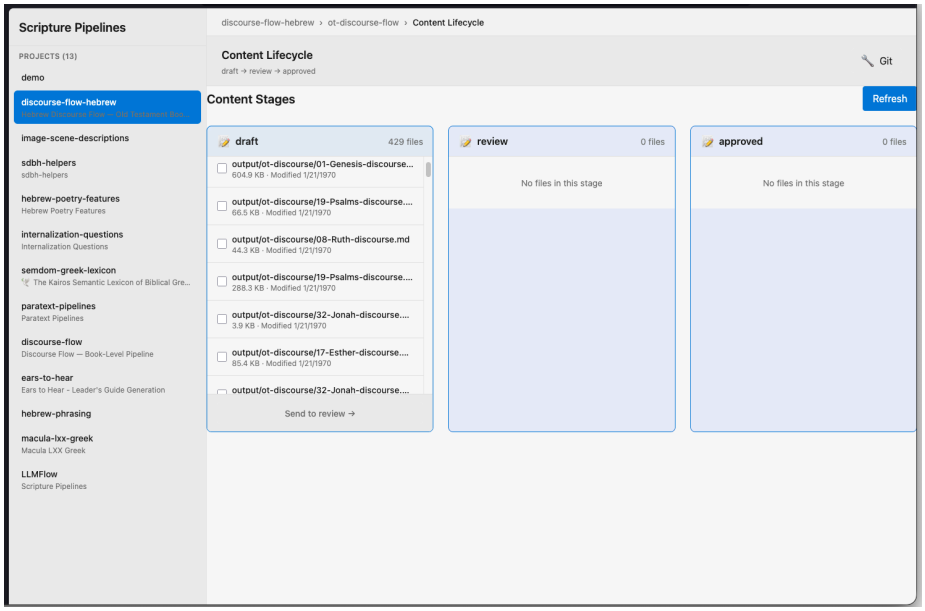

Figure 13: Content Lifecycle dashboard showing stage transitions

`llmflow`) is all that's needed. The `sp` command becomes globally available, but its dependencies remain isolated.

3. **Bundled GUI assets.** The React frontend is pre-built and bundled in the distribution. End users never need Node.js, npm, or frontend build tools. The Flask server serves static assets from its package data.

This deployment model is critical for Scripture Pipelines' target audience. Ministry leaders should not need to understand Python packaging, virtual environments, or frontend build systems. They install once, run `sp gui`, and get a working web interface with no additional configuration.

For advanced users and developers, the CLI remains primary. The GUI is a convenience layer, not a replacement for the pipeline language. All GUI operations map directly to CLI commands and YAML configurations, ensuring feature parity and avoiding lock-in to the GUI workflow.

Related Work

XProc

XProc is the closest structural parallel to Scripture Pipelines: both are declarative pipeline languages that define a graph of steps with named ports, typed inputs and outputs, and compound step composition. XProc 3.0's extension to non-XML document types, its `p:for-each` step over sequences, and its atomic/compound step model are all recognizable cousins of Scripture Pipelines constructs.

The fundamental difference is the document model. XProc pipelines process documents flowing through ports; the unit of computation is a document transformation. Scripture Pipelines pipelines generate knowledge by passing structured context to language models; the unit of computation is a prompt with a contract and a JSON response. XProc has no concept of a prompt, a token budget, a model temperature, or a retry condition on model output quality. Scripture Pipelines has no concept of a serialization pipeline or namespace-aware document trees.

In practice the two are complementary. Scripture Pipelines handles the LLM-generation phase; XProc handles downstream document transformation. The Discourse Flow pipeline, for example, produces USX as one of its output formats — that USX is a natural input to an XProc pipeline for subsequent publishing transforms. Similarly, TEI documents produced by scholarly editors are natural inputs to Scripture Pipelines function steps that prepare context for LLM analysis. TEI's milestone elements, standoff annotations, and `<interp>` infrastructure already provide the kind of structured scholarly overlay that Scripture Pipelines pipelines can read, generate, and augment.

LangChain, LlamaIndex, and Haystack

LangChain and LlamaIndex are the dominant Python frameworks for LLM application development. Both are imperative: pipelines are Python code that chains objects together. This gives maximum flexibility but removes the properties that matter most for scholarly workflows — readability by non-programmers, declarative auditability, and separation of pipeline logic from prompt content. A LangChain chain is not something a lexicographer can read and modify. A Scripture Pipelines pipeline is.

Neither framework treats intermediate outputs as first-class persistent artifacts. LangChain's memory and caching mechanisms are designed for chat history, not for the multi-day, multi-pass generation workflows of lexicography or discourse analysis where being able to rerun from step three without repeating steps one and two is essential economics, not a convenience. Neither framework has a concept analogous to prompt contracts: there is no mechanism that prevents a prompt from being called with missing inputs, and no validation before a run begins that the data flow is coherent.

Haystack is closer in spirit — it supports YAML pipeline declarations and has a stronger emphasis on structured pipelines — but its design center is retrieval-augmented generation and document QA, not the mixed deterministic-plus-generative scholarly workflows that Scripture Pipelines targets.

DSPy

DSPy takes a fundamentally different approach: rather than having authors write prompts, it compiles prompt programs from signatures and optimizes them automatically against a metric. This is powerful for tasks with well-defined correctness criteria, but it is antithetical to the requirements of scholarly pipeline work. A lexicographer needs to read, understand, and argue about every instruction given to the model. An automatically compiled prompt is a black box. The Abbott-Smith audit pattern described in the SemLex section — where the prompt explicitly instructs the model to enumerate its sources and record its reasoning — is not expressible in a compiled prompt framework. The audit trail requires human authorship of the prompt.

GATE and UIMA

GATE (General Architecture for Text Engineering) and UIMA (Unstructured Information Management Architecture) are annotation pipeline frameworks with long histories in computational linguistics and NLP. Both support TEI-aware processing configurations and provide rich annotation schemas over document structure. Both treat the pipeline as a graph of processing resources that annotate a document in sequence — a model structurally similar to Scripture Pipelines' step sequence.

The divergence is again in the generation model. GATE and UIMA pipelines analyze and annotate existing text; they do not generate new text or make calls to probabilistic language models as first-class operations. Incorporating an LLM into a GATE pipeline is possible but requires custom Java components and treats the LLM as an external service rather than a native pipeline citizen. Scripture Pipelines inverts this: the LLM call is the primary step type, and deterministic processing is the extension point.

Workflow Orchestration Frameworks

General-purpose workflow orchestration tools — Prefect, Airflow, Luigi — address the scheduling, dependency tracking, and failure recovery of data pipelines at scale. Their DAG model and task dependency declarations are structurally related to Scripture Pipelines' step sequence. However, these frameworks target data engineering workflows executed by developers; they have no concept of a prompt contract, no support for LLM-specific concerns (model selection, token budgets, output type parsing), and no provision for the scholar-facing readability and human editorial loop that Scripture Pipelines is designed around. They are also substantially more complex to install and operate than a single self-contained binary.

Conclusion

This paper opened with a challenge: biblical and linguistic scholarship has accumulated extraordinary open datasets representing decades of scholarly labor, but the bottleneck is human capacity to synthesize that data at scale and turn it into materials that translators, pastors, students, and reading communities can actually use. The barrier is not data availability — most of it is freely licensed — but the infrastructure and workflows needed to bridge the gap between "data exists" and "practitioners can use it."

Scripture Pipelines addresses this challenge through a combination of technical design and collaborative discipline. The technical infrastructure — declarative YAML pipeline definitions, prompt contracts with explicit input/output specifications, structured JSON intermediates with embedded accountability fields, and persisted outputs at every step — makes AI-assisted scholarly work auditable and reproducible. The collaborative framework — the James Kirk model where humans command and AI executes, clear role separation between scholars, pipeline architects, core developers, and end users, and four interlocking disciplines of test-driven development, explain-before-implementing, persistent context files, and cross-repository coordination — keeps human judgment and scholarly accountability at the center.

The case studies reported here demonstrate that this approach works in practice. A multi-pass Greek semantic lexicon synthesizing Abbott-Smith, BDAG, and hundreds of papyrus citations. A Hebrew Bible grammatical phrasing analysis

built on Macula Hebrew treebank data and Levinsohn discourse annotations. Book-level discourse analyses applying the complete LGNTDF and HOTDF datasets to identify pericope boundaries and rhetorical structures. Leaders' guides for Bible study that ground interpretive questions in exegetical analysis rather than paraphrase. None of these were feasible for underresourced projects before. Now a lean team — a pipeline designer, a handful of scholars, and focus groups of real users — can produce freely licensed reference materials grounded in the full breadth of available scholarly data.

The persistence and audit infrastructure described in this paper — saving every intermediate output, making identifier consistency debuggable through checkpoint inspection, enabling grounding audits that trace final outputs back to the specific source data that should have informed them — addresses the singular challenge of LLM-assisted scholarly work: how do you verify that the model actually used the evidence it was given, rather than generating plausible-sounding text from its training distribution? The answer is accountability fields in JSON schemas that force the LLM to enumerate its sources, and checkpoint files that let a human scholar inspect what the model saw at each stage. This is not primarily a technical achievement; it is a discipline that makes the work trustworthy.

The prompt organization patterns described here — dual-audience readability for both humans and AI, YAML frontmatter declaring contracts, hierarchical section structure with clear information scent, GOOD/BAD examples demonstrating patterns rather than word-specific bans, and audit skills that check for LLM-generated training data contamination — solve a practical problem that emerges at scale: how do you maintain complex generation workflows across dozens of sessions without cognitive overload causing both humans and AI to miss critical context? The 500-line prompts processing biblical passages with 50+ input fields are not toy examples; they are production specifications where both audiences must find what they need quickly or fail. The measured impact from reorganizing production prompts — 60% reduction in misalignment errors, 75% faster human audit time, 40% reduction in AI explanation-phase errors — demonstrates that information architecture matters for AI collaboration just as it matters for human-human technical communication.

The format pragmatism throughout Scripture Pipelines — YAML for human-readable configuration, JSON for structured LLM output and intermediate data that must validate against schemas, Markdown for human-facing content with maximum readability, XML where overlapping structures and standardized publishing pipelines require it — reflects a design philosophy that serves scholarly communities better than format purity would. Each format is used where it excels; none is elevated to universal status. The iteration mechanisms — `--stop-after` to inspect output before downstream steps consume it, `--from-step` to resume after fixing one stage without re-running expensive earlier stages, `sp list-contexts` to see what variables are

available at any point — make the pipeline state inspectable and the debugging workflow tractable. These are not incidental conveniences; they are essential infrastructure for scholarly work where verification cannot be deferred to the end.

None of this diminishes the role of human expertise — it transforms how that expertise is applied. A scholar who knows that the Louw-Nida / SemDom question chain can navigate 1,792 semantic domains to a single word's classification in approximately four questions does not need less expertise to work with Scripture Pipelines than without it; they need more. The pipeline amplifies the reach of that insight across an entire corpus. A discourse analyst who recognizes the significance of Levinsohn's word-level annotations for pericope segmentation unlocks a dataset of 806 structural signals in 1 John alone that would otherwise remain invisible to any pipeline that didn't know to look. The dreaming, the problem framing, the choice of which scholarly tradition to ground a pipeline in, the judgment to know when the model's output is wrong, the taste to know what good enough means for a translator in the field — none of that is supplied by the tool. What changes is the leverage: a single scholar with deep domain knowledge and a well-designed pipeline can now produce work that would previously have required a large team and many years. Experience still counts. It just goes further.

XML's Enduring Niche

Many younger engineers view XML as a “weird old Boomer format” that seems unnecessarily complex. This perception reflects a generation gap, not a technical failure. Engineers building REST APIs, deploying microservices, and writing configuration files work in domains where JSON and YAML excel. They have never encountered the problems XML was designed to solve: overlapping hierarchies in literary texts, scholarly apparatus in critical editions, transformation pipelines in publishing workflows, or standardized queries across heterogeneous document collections.

But those problems did not disappear. TEI scholarly editions still require markup for multiple overlapping structures—syntactic, rhetorical, manuscript witness variants—that cannot be represented in JSON's single tree. DITA technical documentation still needs conditional processing, content reuse, and transformation to multiple output formats. Publishing workflows still require XSLT pipelines that have no equivalent in other ecosystems. These domains need XML's complete infrastructure: validation, querying, transformation, and publishing—all standardized, all interoperable.

XML succeeded as a pioneer by proving principles and making mistakes that taught the world what to simplify. It demonstrated that human-readable text could be data at web scale. It proved that Unicode should be everywhere. It showed that structure matters and validation catches errors. And by

overreaching—making schemas too complex, creating specification documents too long, trying to be the universal format—it taught the next generation to build simpler, more focused tools.

The simpler formats that followed are XML's legacy, not its replacement. JSON learned from XML Schema's complexity and standardized on a simpler model. YAML learned from XML's verbosity and eliminated angle brackets. Markdown learned from XHTML's rigidity and made HTML authoring human-friendly. Each format learned lessons that XML taught through two decades of production use.

The final Balisage conference honors XML's achievements while acknowledging this reality: XML did not fail because simpler formats succeeded. XML succeeded by showing what worked, teaching through its mistakes, and enabling the formats that came next—while maintaining its enduring value for the domains whose problems it was designed to solve.

Scripture Pipelines: Infrastructure for The Kairos Vision

Scripture Pipelines exists to serve The Kairos Project — the Nida Institute's initiative to build a global community of scholars serving Bible translation and the global church. When we say "a scholar with deep domain knowledge," we mean scholars worldwide — including those learning Greek and Hebrew outside traditional seminaries, multilingual learners who bring linguistic gifts that Western institutions have not recognized, translation teams who know their target audience in ways the academy never will. The goal is to democratize the tools so that scholarship can happen where it is needed, by people from diverse backgrounds, producing resources together with the communities who will actually use them. Scripture Pipelines is infrastructure for that vision. It makes the open datasets — the treebanks, the lexicons, the discourse annotations — accessible to scholars globally, whether or not they have formal credentials from established institutions. The Kairos Project exists to serve the global church — to help people everywhere read and understand the Bible accurately. Scripture Pipelines makes that possible by connecting scholars, translators, pastors, teachers, and study leaders in overlapping communities where everyone learns from everyone else. Scholars learn from the questions translation teams and pastors ask. Translation teams and pastors learn from scholarly insights. Study leaders learn from both. We are members of one body, serving together, each bringing what we know, each learning from the others.

The projects reported here are early instances of a larger program. The dataset registry and system integrations provide the infrastructure for an expanding range of pipeline-producible scholarly materials. The goal is a world where the open data that scholars have painstakingly assembled becomes the shared foundation for work that communities worldwide do together.

The introduction to this paper argued that we are at a kairos moment — the opportune time when the open data exists, the communities who need it exist, and the capability to synthesize that data at scale exists. The question is not whether we will respond but how: will we build infrastructure that serves human flourishing, or infrastructure that displaces it? Will we build tools that amplify human judgment and accountability, or tools that claim to replace them? The James Kirk model — humans command, AI executes — is not a compromise with current technological limitations. It is a design choice about what kind of world we want to build: one where scholarship remains the work of scholars, where translation builds community, where the work of understanding Scripture together is not something we automate away but something we do better, together, with better tools. Scripture Pipelines exists to make that choice sustainable at scale. The infrastructure described in this paper makes rigorous, reproducible, community-accountable AI-assisted scholarship possible. Whether that possibility becomes reality depends on the scholars, translators, and communities who choose to build with it — and on their commitment to keeping human wisdom and judgment at the center.

A Note on AI in the Writing of This Paper

This paper was written with AI assistance, in a manner consistent with the principles it describes. The author used GitHub Copilot as a collaborator throughout: drafting passages, testing arguments, refining phrasing, and at several points returning to the saved chat histories from specific projects — the SemLex lexicon, the Hebrew Phrasing analysis, the Discourse Flow pipeline, Ears to Hear, and the Scripture Pipelines engine itself — the full record of prompts, responses, and revisions across each collaboration. The author shared those histories with the AI and reflected, project by project, on how the work had actually proceeded: what the human had directed, what the AI had contributed, where miscommunications had occurred and been corrected. Those reflection sessions became the primary source for the human-command section — the author described how the collaborations had gone, and the AI helped put those descriptions into prose that generalized across them.

The James Kirk model applied. The author read everything, accepted or rejected or revised every sentence, and directed every significant turn in the argument. The AI did not decide what the paper should argue; it helped the author say what he had decided to argue. The paper's central claim — that human judgment, scholarly accountability, and community ownership are not transferable to a language model — was not undermined by the process of writing it this way. If anything, the process was an extended test of that claim, and the claim held.

References

Abbott-Smith, G. *A Manual Greek Lexicon of the New Testament*. T&T Clark, Edinburgh, 1937.

Louw, J. P. and E. A. Nida. *Greek-English Lexicon of the New Testament Based on Semantic Domains*, 2 vols. United Bible Societies, New York, 1988.

Levinsohn, S. H. *Discourse Features of New Testament Greek: A Coursebook on the Information Structure of New Testament Greek*, 2nd ed. SIL International, Dallas, 2000.

Levinsohn, S. H. *Levinsohn Greek New Testament Discourse Features (LGNTDF)*. Data set. Biblical Humanities. https://github.com/biblicalhumanities/levinsohn.

Clear Bible. *Macula Greek Linguistic Datasets*. Data set. CC BY 4.0. https://github.com/Clear-Bible/macula-greek.

Clear Bible. *Macula Hebrew Linguistic Datasets*. Data set. CC BY 4.0. https://github.com/Clear-Bible/macula-hebrew.

Levinsohn, S. H. and D. Samuel. *Hebrew Old Testament Discourse Features (HOTDF)*. Data set. Nida Institute. https://github.com/nida-institute/levinsohn-samuel-hebrew.

United Bible Societies. *Semantic Dictionary of Biblical Hebrew (SDBH)*. Data set. Open license. https://github.com/ubsicap/ubs-open-license. https://semanticdictionary.org/.

OpenScriptures. *Hebrew Bible (morphhb): Parsed Hebrew Bible with Strong's numbers and morphology*. Data set. MIT License. https://github.com/openscriptures/morphhb.

Elwell, W. A. and P. W. Comfort (eds.). *Tyndale Bible Dictionary*. Tyndale House Publishers, Carol Stream, IL, 2001.

Raasveldt, M. and H. Mühleisen. "DuckDB: an Embeddable Analytical Database." *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, pp. 1981–1984, 2019.

Holmes, M. W. (ed.). *The Greek New Testament: SBL Edition*. Society of Biblical Literature and Logos Bible Software, Atlanta and Bellingham, 2010. Custom license. http://sblgnt.com/. https://github.com/morphgnt/sblgnt.

Bunning, A. *Center for New Testament Restoration: Transcriptions of Greek New Testament Manuscripts*. https://greekcntr.org/.

Brannan, R. *ACAI*. Mission Mutual, 2025. CC BY-SA 4.0. Dataset. https://github.com/BibleAquifer/ACAI.

USFM/X Technical Committee. *USFM/USX Technical Documentation*. https://github.com/usfm-bible/tcdocs.

Scripture Burrito Taskforce. *Scripture Burrito: A Specification for Bible-Containing Containers*. https://docs.burrito.bible/.

Grün, C. et al. *BaseX: The XML Database*. https://basex.org/.

Walsh, N. et al. *XProc 3.0: An XML Pipeline Language*. W3C Working Group Note, 2022. https://spec.xproc.org/3.0/xproc/.

Nida Institute. *Awesome Biblical Data: A Curated Catalog of Datasets and Tools for Biblical and Linguistic Scholarship*. 2026–. CC BY-SA 4.0. https://github.com/nida-institute/awesome-biblical-data.

Chase, H. et al. *LangChain*. 2022–. https://www.langchain.com/.

Liu, J. et al. *LlamaIndex*. 2022–. https://www.llamaindex.ai/.

deepset. *Haystack: Open-Source LLM Framework*. 2020–. https://haystack.deepset.ai/.

Khattab, O. et al. "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines." arXiv:2310.03714, 2023.

Cunningham, H. et al. *Developing Language Processing Components with GATE (a User Guide)*. University of Sheffield, 2011. https://gate.ac.uk/.

Ferrucci, D. and A. Lally. "UIMA: An Architectural Approach to Unstructured Information Processing in the Corporate Research Environment." *Natural Language Engineering* 10(3–4):327–348, 2004.

papyri.info: Integrating Digital Papyrology. Duke Databank of Documentary Papyri and Heidelberger Gesamtverzeichnis der griechischen Papyrusurkunden Ägyptens. https://papyri.info/.

Crane, G. (ed.). *Perseus Digital Library*. Tufts University. https://www.perseus.tufts.edu/.

Open Greek and Latin Project. *First1KGreek: First Thousand Years of Greek*. https://opengreekandlatin.github.io/First1KGreek/.

EpiDoc Collaborative. *EpiDoc: Epigraphic Documents in TEI XML*. https://epidoc.stoa.org/.

Robie, J. *Ears to Hear*. Nida Institute, 2025. https://github.com/nida-institute/ears-to-hear.

Chafe, Wallace. *Discourse, Consciousness, and Time: The Flow and Displacement of Conscious Experience in Speaking and Writing*. University of Chicago Press, Chicago, 1994.

Robie, J. "Artificial Intelligence and Bible Translation." *Bible History Daily*. Biblical Archaeology Society, January 3, 2024. https://www.biblicalarchaeology.org/daily/artificial-intelligence-and-bible-translation/.

Bosak, J. “XML, Java, and the future of the Web.” *World Wide Web Journal* 2(4), 1997. https://www.xml.com/pub/a/w3j/s3.bosak.html.

Sperberg-McQueen, C. M. “Markup systems and the future of scholarly text processing.” *Communications of the ACM* 34(11):63–76, 1991.

Sperberg-McQueen, C. M. and C. Huitfeldt. “GODDAG: A Data Structure for Overlapping Hierarchies.” *DDEP/PODDP 2000: Principles of Digital Document Processing*, pp. 139–160. Springer, Berlin, 2000.

Sperberg-McQueen, C. M., C. Huitfeldt, and A. Renear. “Markup and Meaning: What's a Text Really Made Of?” (aka ”Documents as LMNL”). Delivered at ALLC/ACH 2004, Gothenburg. Revised 2007. http://www.tei-c.org/Vault/Talks/2004/lmnl.html.

Burnard, L. and C. M. Sperberg-McQueen (eds.). *TEI P5: Guidelines for Electronic Text Encoding and Interchange.* Text Encoding Initiative Consortium, 2007–. https://tei-c.org/guidelines/p5/.

Sperberg-McQueen, C. M. “Rabbit/Duck Grammars: A Characterization of Overlap.” *Proceedings of Balisage: The Markup Conference 2013*. Balisage Series on Markup Technologies, vol. 10. doi:10.4242/BalisageVol10.Sperberg-McQueen01.

Buneman, P., S. Davidson, G. Hillebrand, and D. Suciu. “A Query Language and Optimization Techniques for Unstructured Data.” *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, pp. 505–516, 1996.

Abiteboul, S. “Querying Semi-Structured Data.” *Proceedings of the 6th International Conference on Database Theory (ICDT)*, pp. 1–18. Springer, Berlin, 1997.

Abiteboul, S., D. Quass, J. McHugh, J. Widom, and J. Wiener. “The Lorel Query Language for Semistructured Data.” *International Journal on Digital Libraries* 1(1):68–88, 1997.

Goldman, R. and J. Widom. “DataGuides: Enabling Query Formulation and Optimization in Semistructured Databases.” *Proceedings of the 23rd VLDB Conference*, pp. 436–445, Athens, Greece, 1997.

Deutsch, A., M. Fernández, D. Florescu, A. Levy, and D. Suciu. “A Query Language for XML.” *Computer Networks* 31(11–16):1155–1169, 1999. (Proceedings of the 8th International World Wide Web Conference).

Chamberlin, D., J. Robie, and D. Florescu. “Quilt: An XML Query Language for Heterogeneous Data Sources.” *Revised Papers from WebDB 2000: Third International Workshop on the Web and Databases*, pp. 53–62. Springer, Berlin, 2000.

Chamberlin, D., J. Robie, and D. Florescu. “XQuery: A Query Language for XML.” *Proceedings of W3C QL'98 Query Language Workshop*, 1998. Updated

version presented at WWW9 Conference, 2000. https://www.w3.org/TandS/QL/QL98/pp/xquery.html.

Clark, J. and M. Murata. *RELAX NG Tutorial*. OASIS Committee Specification, 3 December 2001. https://relaxng.org/tutorial-20011203.html.

Katz, H. (ed.). *XQuery from the Experts: A Guide to the W3C XML Query Language*. Addison-Wesley, Boston, 2004.

Boag, S., D. Chamberlin, M. F. Fernández, D. Florescu, J. Robie, and J. Siméon (eds.). *XQuery 1.0: An XML Query Language*. W3C Recommendation, 23 January 2007. https://www.w3.org/TR/xquery/.

Fernández, M., A. Malhotra, J. Marsh, M. Nagy, and N. Walsh (eds.). *XQuery 1.0 and XPath 2.0 Data Model (XDM)*. W3C Recommendation, 23 January 2007. https://www.w3.org/TR/xpath-datamodel/.

Chamberlin, D., J. Robie, and D. Florescu (eds.). *XML Query Use Cases*. W3C Working Group Note, 23 March 2007. https://www.w3.org/TR/xquery-use-cases/.

Robie, J. "XML Processing and Data Integration with XQuery." *IEEE Internet Computing* 11(4):62–67, July–August 2007. doi:10.1109/MIC.2007.96.

Florescu, D. and G. Fourny. "JSONiq: The History of a Query Language." *IEEE Internet Computing* 17(5):86–90, 2013.

Chamberlin, D. and J. Robie (eds.). *XQuery 3.1 Use Cases*. W3C Working Group Note, 8 December 2015. https://www.w3.org/TR/xquery-31-use-cases/.

Robie, J., M. Dyck, and J. Spiegel (eds.). *XML Path Language (XPath) 3.1*. W3C Recommendation, 21 March 2017. https://www.w3.org/TR/xpath-31/.

Robie, J., M. Dyck, and J. Spiegel (eds.). *XQuery 3.1: An XML Query Language*. W3C Recommendation, 21 March 2017. https://www.w3.org/TR/xquery-31/.

Wright, A., H. Andrews, B. Hutton, and G. Dennis (eds.). *JSON Schema: A Media Type for Describing JSON Documents*. Internet-Draft draft-bhutton-json-schema-01, December 2022. https://json-schema.org/.

Couchbase, Inc. *N1QL: A Query Language for JSON*. https://www.couchbase.com/products/n1ql. https://docs.couchbase.com/server/current/n1ql/n1ql-language-reference/.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. "Attention is All You Need." *Advances in Neural Information Processing Systems 30* (NeurIPS 2017).

Alammar, Jay. "The Illustrated Transformer." 2018. https://jalammar.github.io/illustrated-transformer/.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. "Language Models are Few-Shot Learners." *Advances in Neural Information Processing Systems 33* (NeurIPS 2020).

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *Advances in Neural Information Processing Systems 35* (NeurIPS 2022).

Russell, Stuart. *Human Compatible: Artificial Intelligence and the Problem of Control*. Viking, New York, 2019.